Dr. D. Y. Patil Pratishthan's

DR. D. Y. PATIL INSTITUTE OF ENGINEERING, MANAGEMENT & RESEARCH

Approved by A.I.C.T.E, New Delhi , Maharashtra State Government, Affiliated to Savitribai Phule Pune University
Sector No. 29, PCNTDA , Nigidi Pradhikaran, Akurdi, Pune 411044. Phone: 020–27654470, Fax: 020-27656566
Website : www.dypiemr.ac.in Email : principal.dypiemr@gmail.com

---

DEPARTMENT OF COMPUTER ENGINEERING

LAB MANUAL
Lab Practices V
(Final Year Engineering)
Semester – II

Institute Vision

To strive for excellence by providing quality technical education and facilitate research for the welfare of society

Institute Mission

1. To educate students with strong fundamentals by providing conducive environment
2. To inculcate research with creativity & innovation
3. To strengthen leadership, team work, professional & communication skills and ethical standards
4. To promote Industry Institute collaboration & prepare students for life long learning in context of technological change.

Department Vision

To produce quality computer professionals and fostering research aptitude for dispensing service to society

Department Mission

1. To promote growth of an individual by imparting comprehensive knowledge of tools and technologies.
2. To facilitate research and innovation by engaging faculty and students in research activities.
3. To enrich industry-institute interaction in order to provide a platform to know industry demands and motivation for self-employment.
4. To bring forth a conducive environment to enhance soft skills and professional skills to cater needs of society

Program Specific Outcomes

1. Professional Skills: The ability to comprehend, analyze and develop software and hardware systems and amp; applications through research, in varying domains.

2. Problem-Solving Skills: The ability to apply standard paradigms and strategies in software project development using open-ended programming environments to deliver a quality product.

3. Successful Career and Entrepreneurship: Adaptation of modern practical and systematic approaches in creating innovative solutions for a successful career, entrepreneurship, and a zest for higher studies.

Course Objective

1. To understand and implement searching and sorting algorithms.
2. To learn the fundamentals of GPU Computing in the CUDA environment.
3. To illustrate the concepts of Artificial Intelligence/Machine Learning(AI/ML).
4. To understand Hardware acceleration.
5. To implement different deep learning models.

Course Outcome

After completion of the course, students will be able to :

CO1: Analyze and measure performance of sequential and parallel algorithms.

CO2: Design and Implement solutions for multicore/Distributed/parallel environment.

CO3: Identify and apply the suitable algorithms to solve AI/ML problems.

CO4: Apply the technique of Deep Neural network for implementing Linear regression and classification.

CO5: Apply the technique of Convolution (CNN) for implementing Deep Learning models.

CO6: Design and develop Recurrent Neural Network (RNN) for prediction

@The CO-PO Mapping Matrix												
CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	1	-	1	1	-	2	1	-	-	-	-	-
CO2	1	2	1	-	-	1	-	-	-	-	-	1
CO3	-	1	1	1	1	1	-	-	-	-	-	-
CO4	3	3	3	-	3	-	-	-	-	-	-	-
CO5	3	3	3	3	3	-	-	-	-	-	-	-
CO6	3	3	3	3	3	-	-	-	-	-	-	-
CO7	3	3	3	3	3		-	-	-	-	-	-

Guidelines for Students

The laboratory assignments are to be submitted by student in the form of journal. Journal may home Home Faculty of Engineering Savitribai Phule Pune University Syllabus for Fourth Year of Computer Engineering ` #106/128 consists of prologue, Certificate, table of contents, and handwritten write-up of each assignment (Title, Objectives, Problem Statement, Outcomes, software and Hardware requirements, Date of Completion, Assessment grade/marks and assessor's sign, Theory- Concept in brief, Algorithm/Database design, test cases, conclusion/analysis). Program codes with sample output of all performed assignments are to be submitted as softcopy.

Guidelines for Laboratory /Term Work Assessment

Continuous assessment of laboratory work is to be done based on overall performance and lab assignments performance of student. Each lab assignment assessment will assign grade/marks based on parameters with appropriate weightage. Suggested parameters for overall assessment as well as each lab assignment assessment include- timely completion, performance, innovation, efficient codes, punctuality and neatness reserving weightage for successful mini-project completion and related documentation.

Table of Contents

Group A

Assignment No: 1(A)

Title of the Assignment: Design and implement Parallel Breadth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS

Objective of the Assignment: Students should be able to perform Parallel Breadth First Search based on existing algorithms using OpenMP

Prerequisite:

1. Basic of programming language
2. Concept of BFS
3. Concept of Parallelism

Contents for Theory:

1. **What is BFS?**
2. **Example of BFS**
3. **Concept of OpenMP**
4. **How Parallel BFS Work**
5. **Code Explanation with Output**

What is BFS?

BFS stands for Breadth-First Search. It is a graph traversal algorithm used to explore all the nodes of a graph or tree systematically, starting from the root node or a specified starting point, and visiting all the neighboring nodes at the current depth level before moving on to the next depth level.

The algorithm uses a queue data structure to keep track of the nodes that need to be visited, and marks each visited node to avoid processing it again. The basic idea of the BFS algorithm is to visit all the nodes at a given level before moving on to the next level, which ensures that all the nodes are visited in breadth-first order.

BFS is commonly used in many applications, such as finding the shortest path between two nodes, solving puzzles, and searching through a tree or graph.

Example of BFS

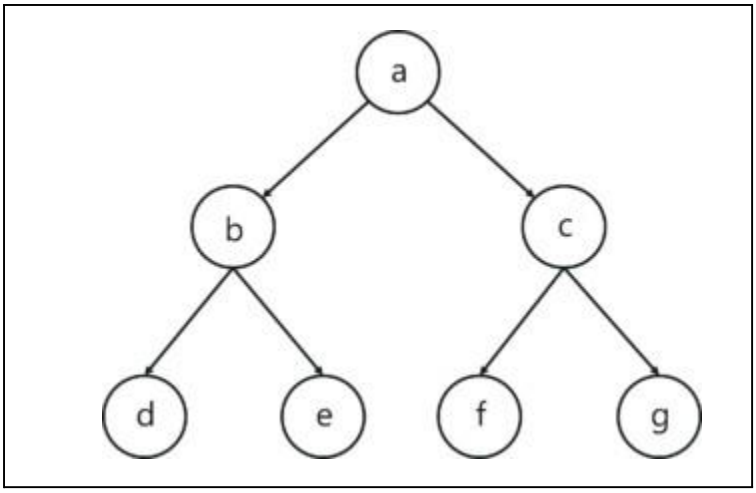
Now let's take a look at the steps involved in traversing a graph by using Breadth-First Search:

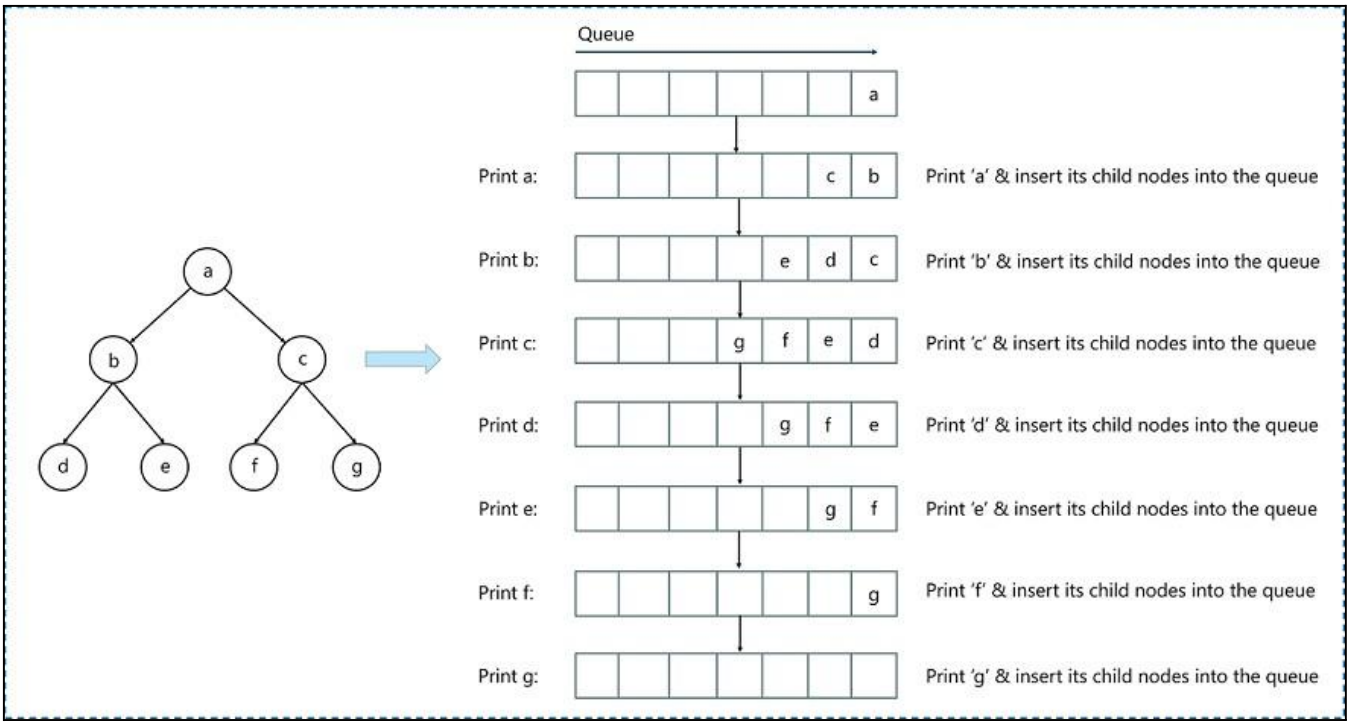
Step 1: Take an Empty Queue.

Step 2: Select a starting node (visiting a node) and insert it into the Queue.

Step 3: Provided that the Queue is not empty, extract the node from the Queue and insert its child nodes (exploring a node) into the Queue.

Step 4: Print the extracted node.





Concept of OpenMP

- OpenMP (Open Multi-Processing) is an application programming interface (API) that supports shared-memory parallel programming in C, C++, and Fortran. It is used to write parallel programs that can run on multicore processors, multiprocessor systems, and parallel computing clusters.
- OpenMP provides a set of directives and functions that can be inserted into the source code of a program to parallelize its execution. These directives are simple and easy to use, and they can be applied to loops, sections, functions, and other program constructs. The compiler then generates parallel code that can run on multiple processors concurrently.
- OpenMP programs are designed to take advantage of the shared-memory architecture of modern processors, where multiple processor cores can access the same memory. OpenMP uses a fork-join model of parallel execution, where a master thread forks multiple worker threads to execute a parallel region of the code, and then waits for all threads to complete before continuing with the sequential part of the code.
- OpenMP is widely used in scientific computing, engineering, and other fields that require high-performance computing. It is supported by most modern compilers and is available on a wide range of platforms, including desktops, servers, and supercomputers.

How Parallel BFS Work

- Parallel BFS (Breadth-First Search) is an algorithm used to explore all the nodes of a graph or tree

systematically in parallel. It is a popular parallel algorithm used for graph traversal in distributed computing, shared-memory systems, and parallel clusters.

- The parallel BFS algorithm starts by selecting a root node or a specified starting point, and then assigning it to a thread or processor in the system. Each thread maintains a local queue of nodes to be visited and marks each visited node to avoid processing it again.
- The algorithm then proceeds in levels, where each level represents a set of nodes that are at a certain distance from the root node. Each thread processes the nodes in its local queue at the current level, and then exchanges the nodes that are adjacent to the current level with other threads or processors. This is done to ensure that the nodes at the next level are visited by the next iteration of the algorithm.
- The parallel BFS algorithm uses two phases: the computation phase and the communication phase. In the computation phase, each thread processes the nodes in its local queue, while in the communication phase, the threads exchange the nodes that are adjacent to the current level with other threads or processors.
- The parallel BFS algorithm terminates when all nodes have been visited or when a specified node has been found. The result of the algorithm is the set of visited nodes or the shortest path from the root node to the target node.
- Parallel BFS can be implemented using different parallel programming models, such as OpenMP, MPI, CUDA, and others. The performance of the algorithm depends on the number of threads or processors used, the size of the graph, and the communication overhead between the threads or processors.

Conclusion- In this way we can achieve parallelism while implementing BFS

Assignment Question

1. **What if BFS?**
2. **What is OpenMP?What is its significance in parallel programming?**
3. **Write down applications of Parallel BFS**
4. **How can BFS be parallelized using OpenMP? Describe the parallel BFS algorithm using OpenMP.**
5. **Write Down Commands used in OpenMP?**

Reference link

- https://www.edureka.co/blog/breadth-first-search-algorithm/

```
#include<iostream>
#include<stdlib.h>
#include<queue>
using namespace std;


class node
{
  public:

   node *left, *right;
   int data;

};

class Breadthfs
{

 public:

 node *insert(node *, int);
 void bfs(node *);

};


node *insert(node *root, int data)
// inserts a node in tree
{

   if(!root)
   {

         root=new node;
         root->left=NULL;
         root->right=NULL;
         root->data=data;
         return root;
```

```
}

queue<node *> q;
q.push(root);

while(!q.empty())
{

    node *temp=q.front();
    q.pop();

    if(temp->left==NULL)
    {

        temp->left=new node;
        temp->left->left=NULL;
        temp->left->right=NULL;
        temp->left->data=data;
        return root;
    }
    else
    {

    q.push(temp->left);

    }

    if(temp->right==NULL)
    {

        temp->right=new node;
        temp->right->left=NULL;
        temp->right->right=NULL;
        temp->right->data=data;
        return root;
    }
    else
    {

    q.push(temp->right);
```

```
        }

    }

}


void bfs(node *head)
{

        queue<node*> q;
        q.push(head);

        int qSize;

        while (!q.empty())
        {
                qSize = q.size();
                #pragma omp parallel for
                //creates parallel threads
                for (int i = 0; i < qSize; i++)
                {
                        node* currNode;
                        #pragma omp critical
                        {
                          currNode = q.front();
                          q.pop();
                          cout<<"\t"<<currNode->data;

                        }// prints parent node
                        #pragma omp critical
                        {
                        if(currNode->left)// push parent's left node in queue
                                q.push(currNode->left);
                        if(currNode->right)
                                q.push(currNode->right);
                        }// push parent's right node in queue

                }
```

```
        }

}

int main(){

   node *root=NULL;
   int data;
   char ans;

   do
   {
        cout<<"\n enter data=>";
        cin>>data;

        root=insert(root,data);

        cout<<"do you want insert one more node?";
        cin>>ans;

   }while(ans=='y'||ans=='Y');

   bfs(root);

   return 0;
}
```

Run Commands:

1) g++ -fopenmp bfs.cpp -o bfs

2) ./bfs

Output:

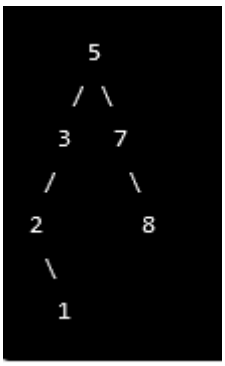
This code represents a breadth-first search (BFS) algorithm on a binary tree using OpenMP for parallelization. The program asks for user input to insert nodes into the binary tree and then performs the BFS algorithm using multiple threads. Here's an example output for a binary tree with nodes 5, 3, 2, 1, 7, and 8:

```
Enter data => 5
Do you want to insert one more node? (y/n) y

Enter data => 3
Do you want to insert one more node? (y/n) y

Enter data => 2
Do you want to insert one more node? (y/n) y

Enter data => 1
Do you want to insert one more node? (y/n) y

Enter data => 7
Do you want to insert one more node? (y/n) y

Enter data => 8
Do you want to insert one more node? (y/n) n

        5       3       7       2       1       8
```

The nodes are printed in breadth-first order. The #pragma omp parallel for statement is used to parallelize the for loop that processes each level of the binary tree. The #pragma omp critical statement is used to synchronize access to shared data structures, such as the queue that stores the nodes of the binary tree.

Here is an example of the breadth-first traversal for a binary tree with the values 5, 3, 2, 1, 7, and 8:

Starting with the root node containing value 5:



The traversal would be:

```
5, 3, 7, 2, 8, 1
```

Explanation

This C++ code demonstrates how to perform a breadth-first search (BFS) in a binary tree using OpenMP parallel programming.

1. The program starts by defining a class "node" that defines the properties of a binary tree node. This class has two pointers to the left and right child nodes of the current node, and an integer to store the data value of the node.
2. Next, a class named "Breadthfs" is defined, which contains two methods - insert() and bfs(). The insert() method is used to insert a new node in the binary tree, while the bfs() method is used to perform the BFS algorithm on the binary tree.
3. The insert() method takes two arguments - a pointer to the root node of the binary tree and an integer value to be inserted. If the root node is null, the

method creates a new node, sets its data value to the given integer value and returns the root node.

4. If the root node is not null, the method creates an empty queue of node pointers and pushes the root node into the queue. It then enters a loop that runs until the queue is empty.
5. Inside the loop, the method dequeues the front node from the queue and checks if its left child is null. If it is null, the method creates a new node, sets its data value to the given integer value, and returns the root node.
6. If the left child of the front node is not null, the method pushes it onto the queue. The method then checks if the right child of the front node is null. If it is null, the method creates a new node, sets its data value to the given integer value, and returns the root node.
7. If the right child of the front node is not null, the method pushes it onto the queue.
8. The bfs() method takes a pointer to the root node of the binary tree as its argument. It creates an empty queue of node pointers and pushes the root node into the queue.
9. It then enters a loop that runs until the queue is empty. Inside the loop, it retrieves the size of the queue and creates an OpenMP parallel region using the "omp parallel for" directive. This directive creates a team of parallel threads to execute the loop body in parallel.

10. Inside the loop body, each thread dequeues a node from the queue using a critical section to ensure that no two threads access the same node simultaneously. It prints the data value of the current node to the console.
11. The thread then checks if the left and right child nodes of the current node are not null. If they are not null, the thread uses another critical section to push the left and right child nodes onto the queue.
12. The main function starts by initializing the root node pointer to null and declaring an integer variable to store the user input.
13. It uses a do-while loop to prompt the user to enter a value to be inserted in the binary tree. If the user enters 'y' or 'Y', the loop continues to accept more input. Otherwise, the loop terminates.
14. For each user input value, the program calls the insert() method to insert a new node in the binary tree.
15. After the user is finished inputting values, the program calls the bfs() method to perform a breadth-first search on the binary tree.
16. Finally, the program returns 0 to indicate successful execution.

Explanation 2

This C++ code implements the Breadth-First Search (BFS) algorithm to traverse a binary tree. Here is a step-by-step explanation of the code's execution flow:

1. The code defines a node class with left, right, and data members, which are pointers to node, pointers to the left and right child nodes, and the data to be stored in each node, respectively.

2. The code defines a Breadthfs class with insert and bfs member functions, which are responsible for inserting a new node into the binary tree and traversing the tree in a breadth-first manner, respectively.
3. The insert function takes two arguments: a pointer to the root node of the tree and an integer value to be inserted. If the root node is NULL, it creates a new node with the given value and returns the new node. Otherwise, it uses a queue to traverse the tree level by level, looking for the first empty child node (either left or right). When an empty node is found, it creates a new node with the given value and returns the root node.
4. The bfs function takes a pointer to the root node of the tree and performs a breadth-first traversal of the tree. It starts by initializing a queue with the root node and a variable to store the size of the queue. Then, it enters a loop that continues until the queue is empty.
5. Inside the loop, it obtains the current size of the queue, and for each node in the queue, it pops the front node and prints its data value. It then adds the node's left and right child nodes to the queue if they exist.
6. To improve the performance of the BFS traversal, the loop that processes each node is parallelized using OpenMP, a library that enables parallel programming in C++. The #pragma omp parallel for directive creates multiple threads that execute the loop iterations in parallel, and the #pragma omp critical directive ensures that only one thread at a time can access the shared resources (i.e., the queue and the console output).

7. Finally, the main function initializes a pointer to the root node and prompts the user to enter integer values to be inserted into the tree. It uses the insert function to create a new node for each value and adds it to the tree. It continues until the user chooses to stop inserting new values. Then, it calls the bfs function to traverse the tree in a breadth-first manner and prints the data values of each node.

Group A

Assignment No: 1(B)

Title of the Assignment: Design and implement Parallel Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for DFS

Objective of the Assignment: Students should be able to perform Parallel Depth First Search based on existing algorithms using OpenMP

Prerequisite:

1. Basic of programming language
2. Concept of DFS
3. Concept of Parallelism

Contents for Theory:

1. **What is DFS?**
2. **Example of DFS**
3. **Concept of OpenMP**
4. **How Parallel DFS Work**

What is DFS?

DFS stands for Depth-First Search. It is a popular graph traversal algorithm that explores as far as possible along each branch before backtracking. This algorithm can be used to find the shortest path between two vertices or to traverse a graph in a systematic way. The algorithm starts at the root node and explores as far as possible along each branch before backtracking. The backtracking is done to explore the next branch that has not been explored yet.

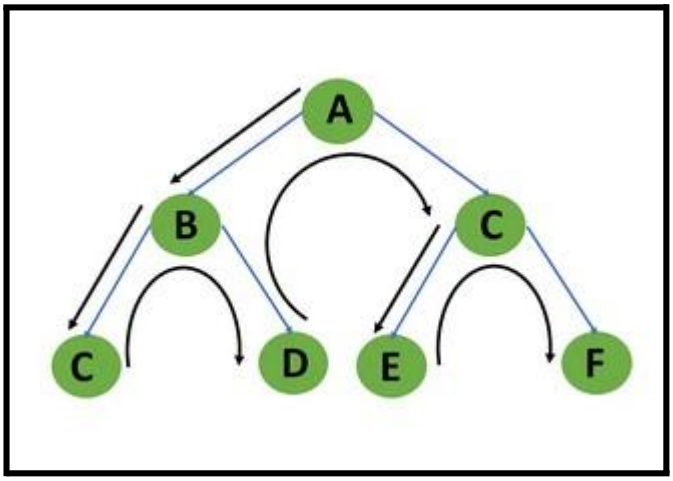
DFS can be implemented using either a recursive or an iterative approach. The recursive approach is simpler to implement but can lead to a stack overflow error for very large graphs. The iterative approach uses a stack to keep track of nodes to be explored and is preferred for larger graphs.

DFS can also be used to detect cycles in a graph. If a cycle exists in a graph, the DFS algorithm will eventually reach a node that has already been visited, indicating that a cycle exists.

A standard DFS implementation puts each vertex of the graph into one of two categories:

1. Visited
2. Not Visited

The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.



Example of DFS:

To implement DFS traversal, you need to take the following stages.

Step 1: Create a stack with the total number of vertices in the graph as the size.

Step 2: Choose any vertex as the traversal's beginning point. Push a visit to that vertex and add it to the stack.

Step 3 - Push any non-visited adjacent vertices of a vertex at the top of the stack to the top of the stack.

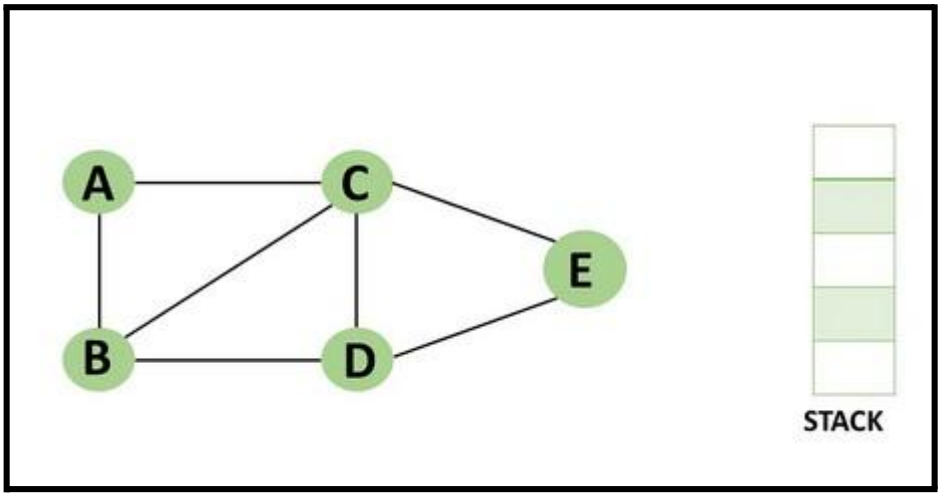
Step 4 - Repeat steps 3 and 4 until there are no more vertices to visit from the vertex at the top of the stack.

Step 5 - If there are no new vertices to visit, go back and pop one from the stack using backtracking.

Step 6 - Continue using steps 3, 4, and 5 until the stack is empty.

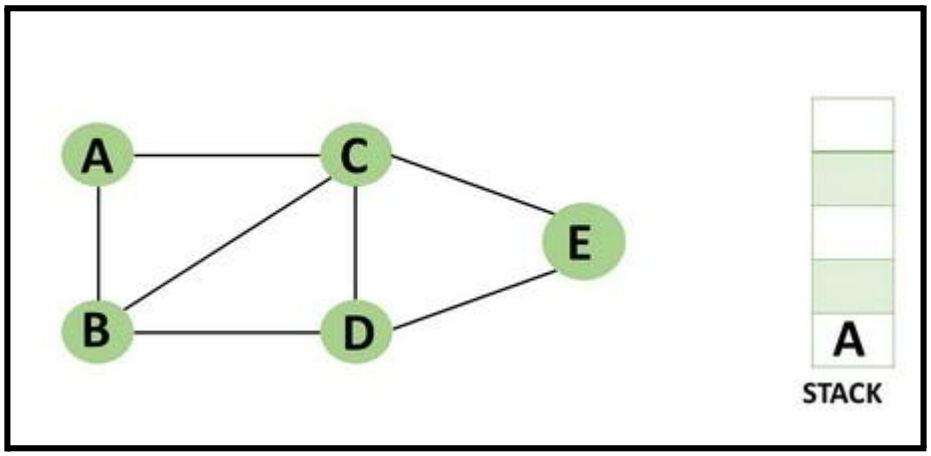
Step 7 - When the stack is entirely unoccupied, create the final spanning tree by deleting the graph's unused edges.

Consider the following graph as an example of how to use the dfs algorithm.



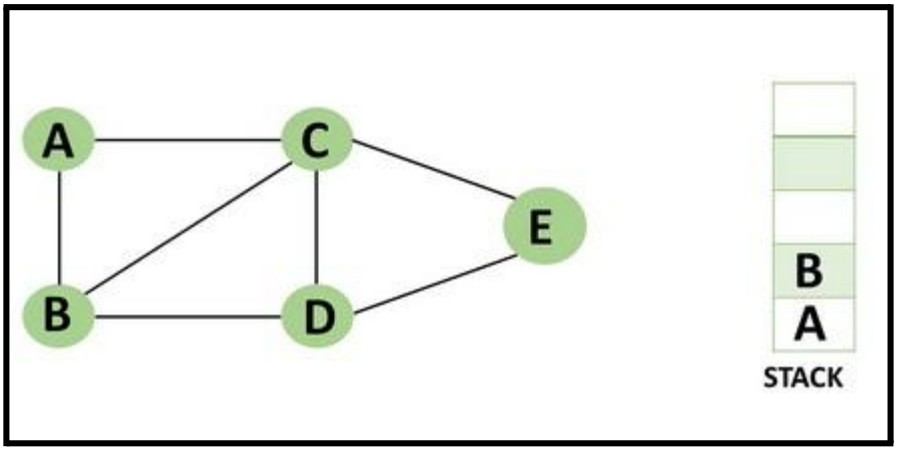
Step 1: Mark vertex A as a visited source node by selecting it as a source node.

- You should push vertex A to the top of the stack.



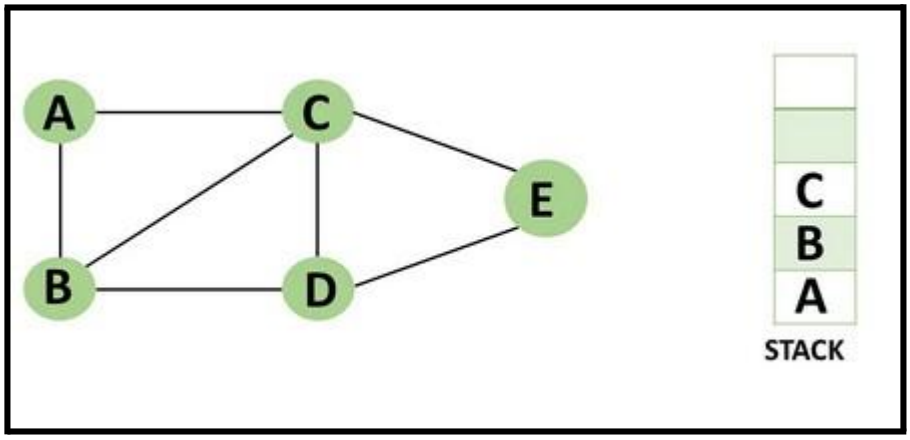
Step 2: Any nearby unvisited vertex of vertex A, say B, should be visited.

You should push vertex B to the top of the stack



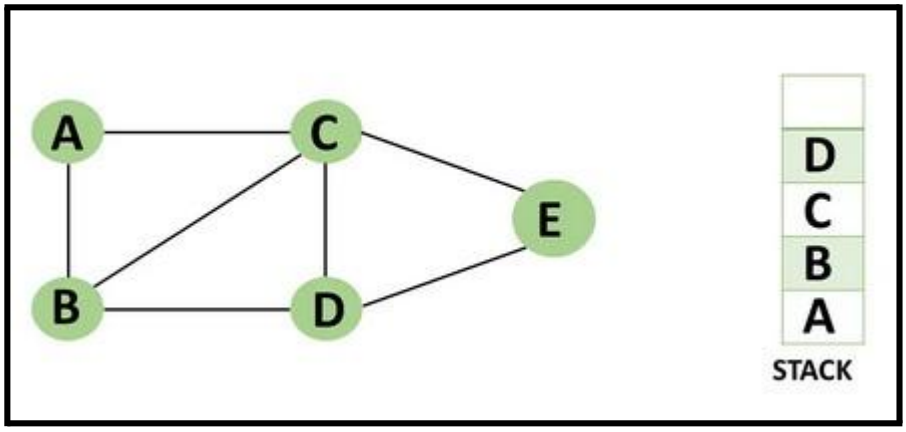
Step 3: From vertex C and D, visit any adjacent unvisited vertices of vertex B. Imagine you have chosen vertex C, and you want to make C a visited vertex.

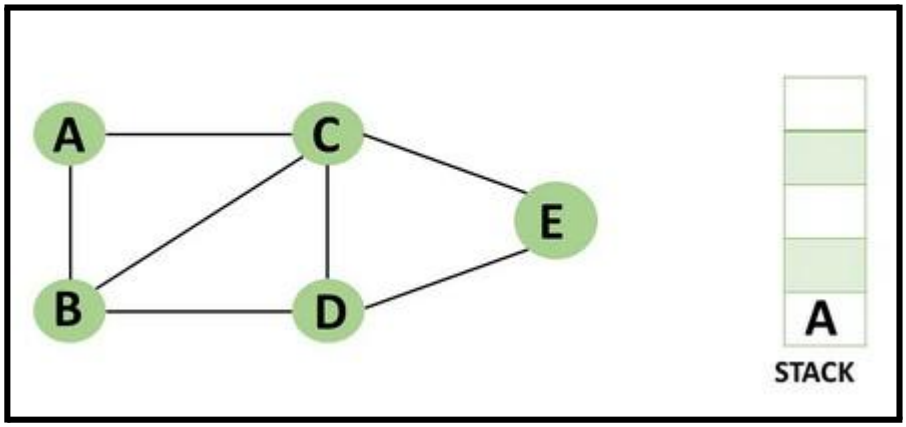
- Vertex C is pushed to the top of the stack.



Step 4: You can visit any nearby unvisited vertices of vertex C, you need to select vertex D and designate it as a visited vertex.

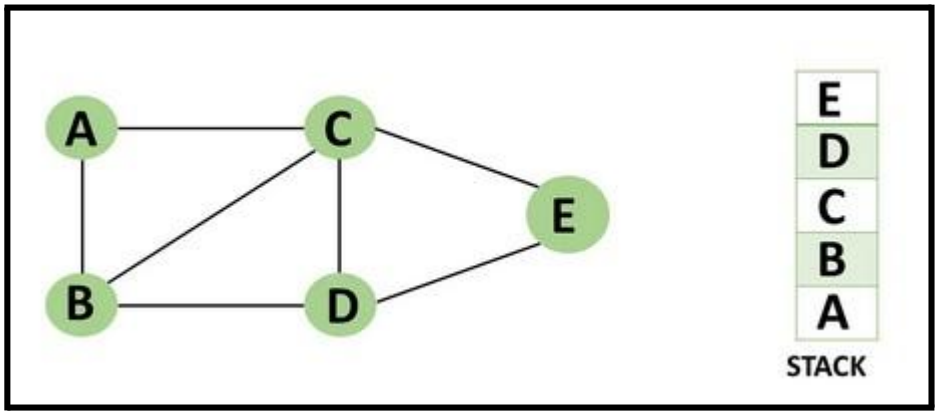
- Vertex D is pushed to the top of the stack.



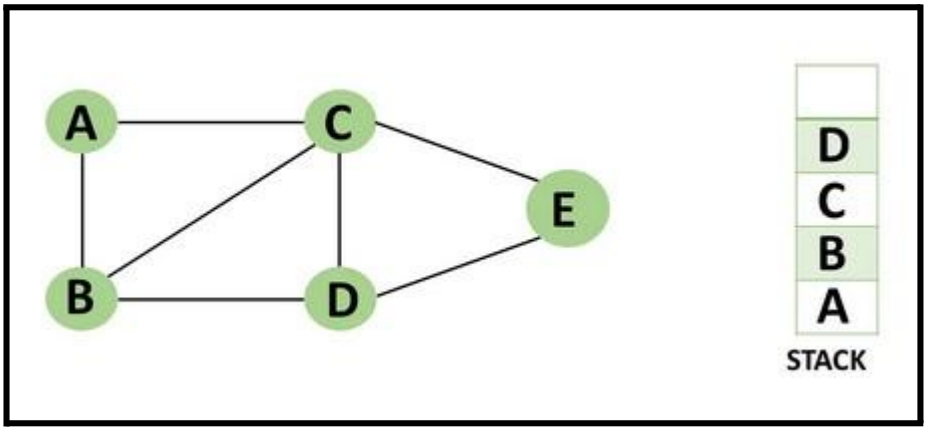


Step 5: Vertex E is the lone unvisited adjacent vertex of vertex D, thus marking it as visited.

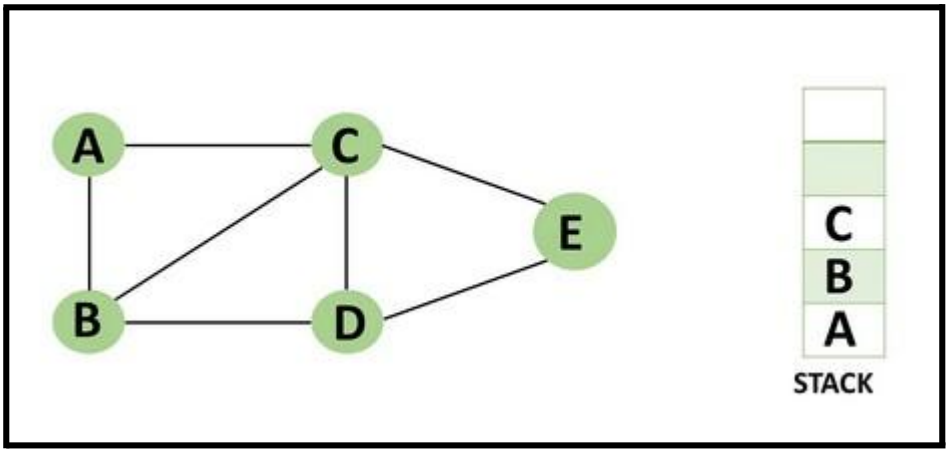
●Vertex E should be pushed to the top of the stack.



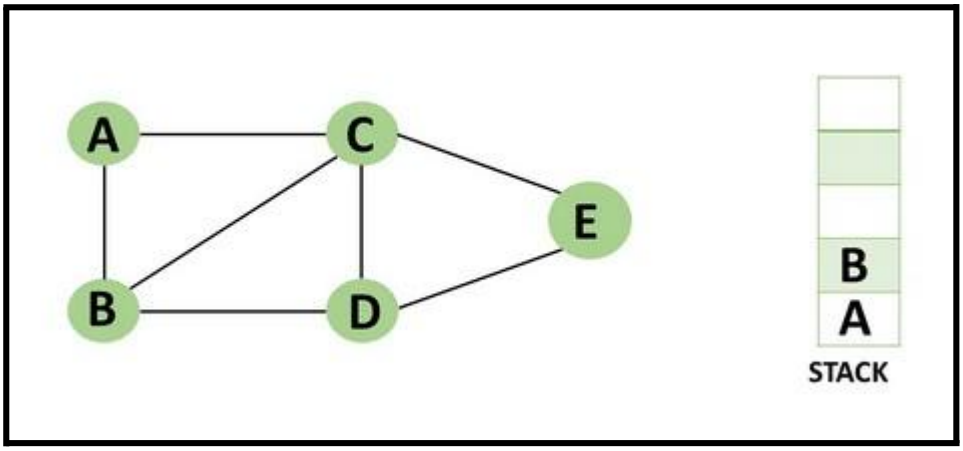
Step 6: Vertex E's nearby vertices, namely vertex C and D have been visited, pop vertex E from the stack.



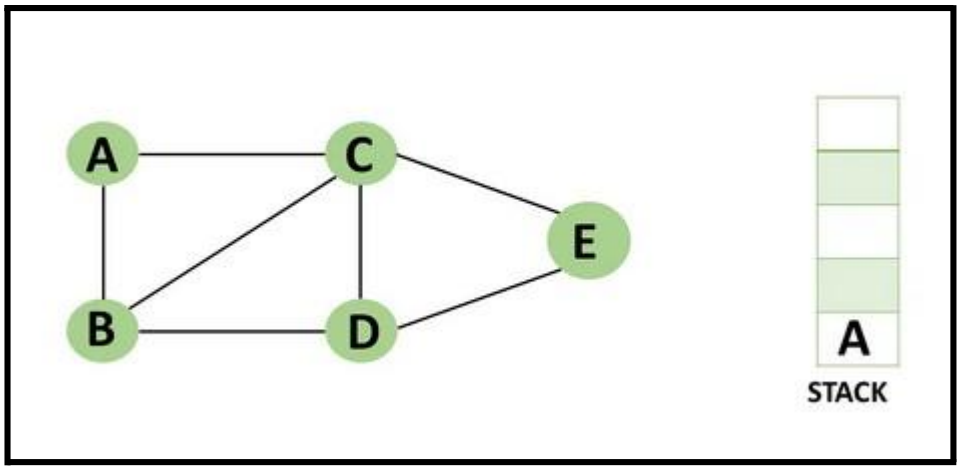
Step 7: Now that all of vertex D's nearby vertices, namely vertex B and C, have been visited, pop vertex D from the stack.



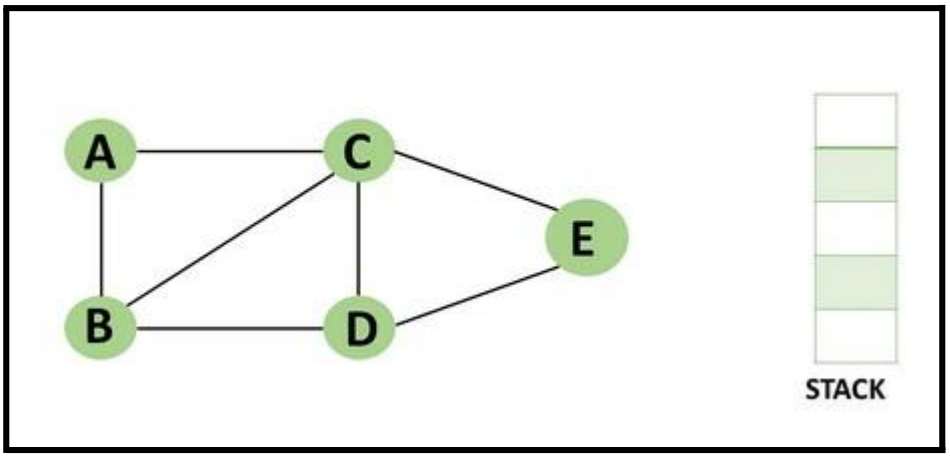
Step 8: Similarly, vertex C's adjacent vertices have already been visited; therefore, pop it from the stack.



Step 9: There is no more unvisited adjacent vertex of b, thus pop it from the stack.



Step 10: All of the nearby vertices of Vertex A, B, and C, have already been visited, so pop vertex A from the stack as well.



Concept of OpenMP

- OpenMP (Open Multi-Processing) is an application programming interface (API) that supports shared-memory parallel programming in C, C++, and Fortran. It is used to write parallel programs that can run on multicore processors, multiprocessor systems, and parallel computing clusters.
- OpenMP provides a set of directives and functions that can be inserted into the source code of a program to parallelize its execution. These directives are simple and easy to use, and they can be applied to loops, sections, functions, and other program constructs. The compiler then generates parallel code that can run on multiple processors concurrently.
- OpenMP programs are designed to take advantage of the shared-memory architecture of modern processors, where multiple processor cores can access the same memory. OpenMP uses a fork-join model of parallel execution, where a master thread forks multiple worker threads to execute a parallel region of the code, and then waits for all threads to complete before continuing with the sequential part of the code.

How Parallel DFS Work

- Parallel Depth-First Search (DFS) is an algorithm that explores the depth of a graph structure to search for nodes. In contrast to a serial DFS algorithm that explores nodes in a sequential manner, parallel DFS algorithms explore nodes in a parallel manner, providing a significant speedup in large graphs.

- Parallel DFS works by dividing the graph into smaller subgraphs that are explored simultaneously. Each processor or thread is assigned a subgraph to explore, and they work independently to explore the subgraph using the standard DFS algorithm. During the exploration process, the nodes are marked as visited to avoid revisiting them.

- To explore the subgraph, the processors maintain a stack data structure that stores the nodes in the order of exploration. The top node is picked and explored, and its adjacent nodes are pushed onto the stack for further exploration. The stack is updated concurrently by the processors as they explore their subgraphs.

- Parallel DFS can be implemented using several parallel programming models such as OpenMP, MPI, and CUDA. In OpenMP, the #pragma omp parallel for directive is used to distribute the work

among multiple threads. By using this directive, each thread operates on a different part of the graph, which increases the performance of the DFS algorithm.

Conclusion- In this way we can achieve parallelism while implementing DFS

Assignment Question

1. **What if DFS?**
2. **Write a parallel Depth First Search (DFS) algorithm using OpenMP**
3. **What is the advantage of using parallel programming in DFS?**
4. **How can you parallelize a DFS algorithm using OpenMP?**
5. **What is a race condition in parallel programming, and how can it be avoided in OpenMP?**

Reference link

- https://www.programiz.com/dsa/graph-dfs
- https://www.simplilearn.com/tutorials/data-structure-tutorial/dfs-algorithm

Assignment 1(b)

Title: Parallel Depth First Search based on existing algorithms using OpenMP

```
#include <iostream>
#include <vector>
#include <stack>
#include <omp.h>

using namespace std;

const int MAX = 100000;
vector<int> graph[MAX];
bool visited[MAX];

void dfs(int node) {
        stack<int> s;
        s.push(node);

        while (!s.empty()) {
        int curr_node = s.top();
        s.pop();

        if (!visited[curr_node]) {
        visited[curr_node] = true;

        if (visited[curr_node]) {
        cout << curr_node << " ";
        }

        #pragma omp parallel for
        for (int i = 0; i < graph[curr_node].size(); i++) {
                int adj_node = graph[curr_node][i];
                if (!visited[adj_node]) {
                s.push(adj_node);
                }
        }
        }
        }
}

int main() {
        int n, m, start_node;
```

```
        cout << "Enter No of Node,Edges,and start node:" ;
        cin >> n >> m >> start_node;
      //n: node,m:edges

cout << "Enter Pair of edges:" ;
        for (int i = 0; i < m; i++) {
        int u, v;

        cin >> u >> v;
//u and v: Pair of edges
        graph[u].push_back(v);
        graph[v].push_back(u);
        }

        #pragma omp parallel for
        for (int i = 0; i < n; i++) {
        visited[i] = false;
        }

        dfs(start_node);

/*      for (int i = 0; i < n; i++) {
        if (visited[i]) {
        cout << i << " ";
        }
        }*/

        return 0;
}
```

Explanation:
Let's go through the code step by step:

1. We start by including the necessary headers and declaring some global variables, such as the graph adjacency list, an array to keep track of visited nodes, and a maximum limit for the number of nodes in the graph.
2. Next, we define a function called dfs() which takes a starting node as input and performs the depth-first search algorithm. We use a stack to keep track of the nodes to be visited. The algorithm works as follows:

- We push the starting node onto the stack.
- While the stack is not empty, we pop the top node from the stack.

- If the current node has not been visited, we mark it as visited and explore all its neighbors that have not been visited yet by adding them to the stack.

3. In the main() function, we read the input values: the number of nodes in the graph, the number of edges, and the starting node.
4. We then read the edge information and build the adjacency list of the graph.
5. We initialize the visited array to false for all nodes in the graph using OpenMP's parallel for construct to allow multiple threads to set the array values in parallel.
6. We call the dfs() function with the starting node, which performs the depth-first search algorithm in parallel.
7. Finally, we print out the list of visited nodes in the order they were visited.

The parallelization of the DFS algorithm is achieved by using the OpenMP parallel for construct inside the dfs() function to explore the neighbours of each node in parallel. This allows multiple threads to work on different parts of the graph simultaneously, which can lead to significant performance gains on multi-core CPUs

step-by-step explanation of how this program works:

dfs function

This function performs a parallel depth-first search starting from a given node. It takes the node number as a parameter and modifies the global visited array to keep track of which nodes have been visited.

1. It starts by creating an empty stack and pushing the starting node onto it.

```c++
void dfs(int node) {
    stack<int> s;
    s.push(node);
```

2. It enters a loop that continues until the stack is empty.

```c++
    while (!s.empty()) {
        int curr_node = s.top();
        s.pop();
```

3. For each node that is popped off the stack, it checks if that node has already been visited. If not, it sets the visited flag for that node to true and adds its unvisited neighbors to the stack in parallel.

```c++
        if (!visited[curr_node]) {
            visited[curr_node] = true;

            #pragma omp parallel for
            for (int i = 0; i < graph[curr_node].size(); i++) {
                int adj_node = graph[curr_node][i];
                if (!visited[adj_node]) {
                    s.push(adj_node);
                }
            }
        }
    }
}
```

Note that the #pragma omp parallel for directive is used to parallelize the loop that adds neighboring nodes to the stack. This can speed up the execution of the program on multi-core machines.

main function

This function reads in the input, initializes the visited array, calls the dfs function, and prints out the list of visited nodes.

1. It reads in the number of nodes, the number of edges, and the starting node from standard input.

```c++
int main() {
    int n, m, start_node;
    cin >> n >> m >> start_node;
```

2. It reads in the edge list and builds the adjacency list representation of the graph. Note that each edge is added to both the u node's and v node's adjacency list to represent an undirected graph.

```c++
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
```

3. It initializes the visited array to false in parallel.

```c++
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        visited[i] = false;
    }
```

4. It calls the dfs function with the starting node as the argument.

```c++
    dfs(start_node);
```

5.It prints out the list of visited nodes in ascending order.

```c++
    for (int i = 0; i < n; i++) {
        if (visited[i]) {
            cout << i << " ";
        }
    }

    return 0;
}
```

Output:
here's an example input and output for a small graph with 6 nodes and 5 edges:

Input:

```
6 7 0
0 1
0 2
1 3
2 4
2 5
4 5
5 3
```

Output:

Copy code

```
0 1 2 4 5 3
```

Group A

Assignment No: 2(A)

Title of the Assignment: Write a program to implement Parallel Bubble Sort. Use existing algorithms and measure the performance of sequential and parallel algorithms.

Objective of the Assignment: Students should be able to Write a program to implement Parallel Bubble Sort and can measure the performance of sequential and parallel algorithms.

Prerequisite:

1. Basic of programming language
2. Concept of Bubble Sort
3. Concept of Parallelism

Contents for Theory:

1. **What is Bubble Sort? Use of Bubble Sort**
2. **Example of Bubble sort?**
3. **Concept of OpenMP**
4. **How Parallel Bubble Sort Work**
5. **How to measure the performance of sequential and parallel algorithms?**

What is Bubble Sort?

Bubble Sort is a simple sorting algorithm that works by repeatedly swapping adjacent elements if they are in the wrong order. It is called "bubble" sort because the algorithm moves the larger elements towards the end of the array in a manner that resembles the rising of bubbles in a liquid.

The basic algorithm of Bubble Sort is as follows:

1. Start at the beginning of the array.
2. Compare the first two elements. If the first element is greater than the second element, swap them.
3. Move to the next pair of elements and repeat step 2.
4. Continue the process until the end of the array is reached.
5. If any swaps were made in step 2-4, repeat the process from step 1.

The time complexity of Bubble Sort is O(n^2), which makes it inefficient for large lists. However, it has the advantage of being easy to understand and implement, and it is useful for educational purposes and for sorting small datasets.

Bubble Sort has limited practical use in modern software development due to its inefficient time complexity of O(n^2) which makes it unsuitable for sorting large datasets. However, Bubble Sort has some advantages and use cases that make it a valuable algorithm to understand, such as:

1. Simplicity: Bubble Sort is one of the simplest sorting algorithms, and it is easy to understand and implement. It can be used to introduce the concept of sorting to beginners and as a basis for more complex sorting algorithms.
2. Educational purposes: Bubble Sort is often used in academic settings to teach the principles of sorting algorithms and to help students understand how algorithms work.
3. Small datasets: For very small datasets, Bubble Sort can be an efficient sorting algorithm, as its overhead is relatively low.
4. Partially sorted datasets: If a dataset is already partially sorted, Bubble Sort can be very efficient. Since Bubble Sort only swaps adjacent elements that are in the wrong order, it has a low number of operations for a partially sorted dataset.
5. Performance optimization: Although Bubble Sort itself is not suitable for sorting large datasets, some of its techniques can be used in combination with other sorting algorithms to optimize their performance. For example, Bubble Sort can be used to optimize the performance of Insertion Sort by reducing the number of comparisons needed.

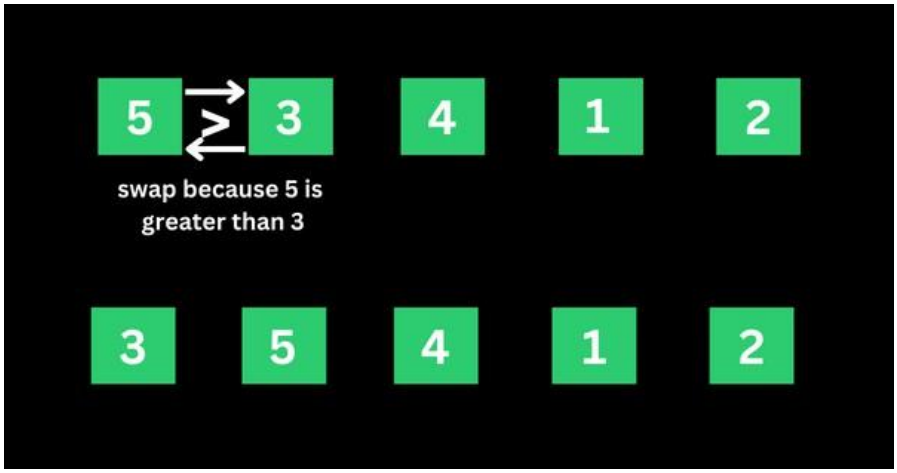
Example of Bubble sort

Let's say we want to sort a series of numbers 5, 3, 4, 1, and 2 so that they are arranged in ascending order…

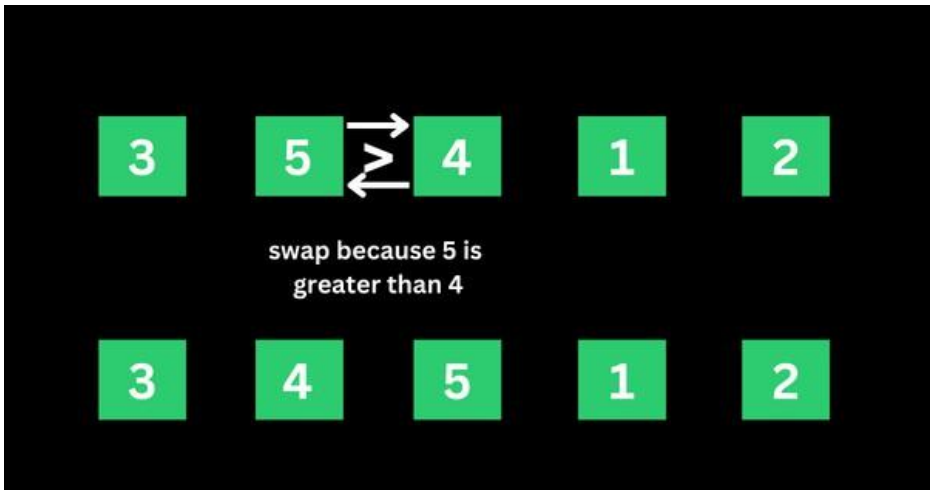
The sorting begins the first iteration by comparing the first two values. If the first value is greater than the second, the algorithm pushes the first value to the index of the second value.

First Iteration of the Sorting

Step 1: In the case of 5, 3, 4, 1, and 2, 5 is greater than 3. So 5 takes the position of 3 and the numbers become 3, 5, 4, 1, and 2.

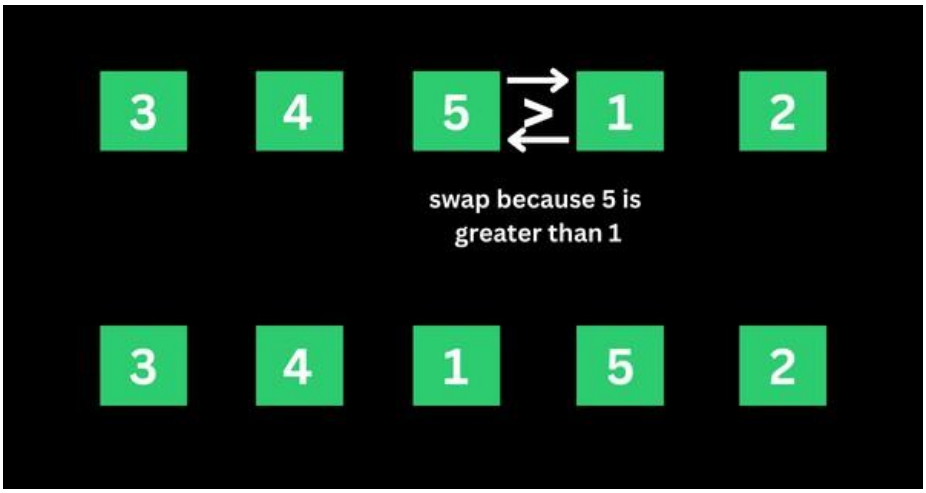


Step 2: The algorithm now has 3, 5, 4, 1, and 2 to compare, this time around, it compares the next two values, which are 5 and 4. 5 is greater than 4, so 5 takes the index of 4 and the values now become 3, 4, 5, 1, and 2.

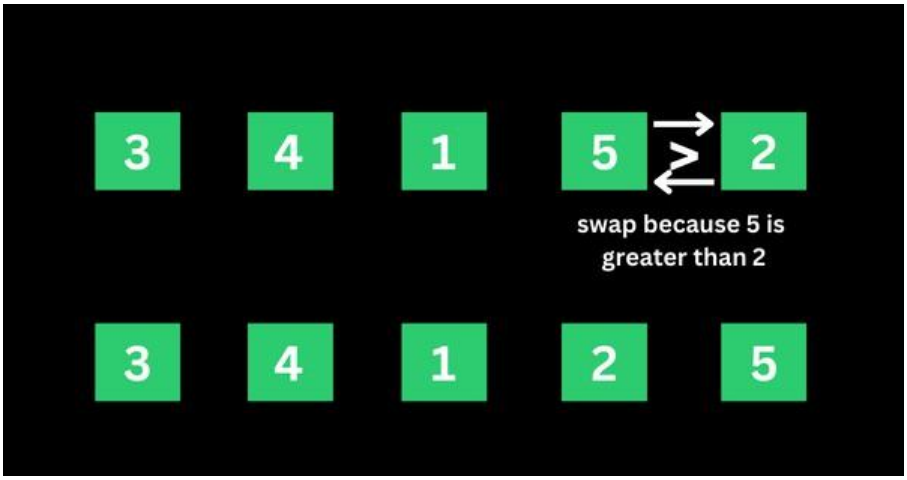


Step 3: The algorithm now has 3, 4, 5, 1, and 2 to compare. It compares the next two values, which are 5

and 1. 5 is greater than 1, so 5 takes the index of 1 and the numbers become 3, 4, 1, 5, and 2.



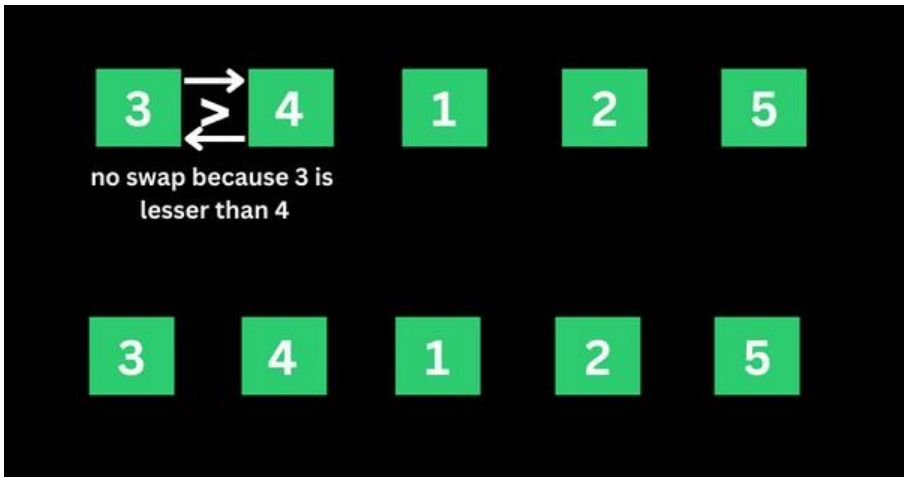
Step 4: The algorithm now has 3, 4, 1, 5, and 2 to compare. It compares the next two values, which are 5 and 2. 5 is greater than 2, so 5 takes the index of 2 and the numbers become 3, 4, 1, 2, and 5.



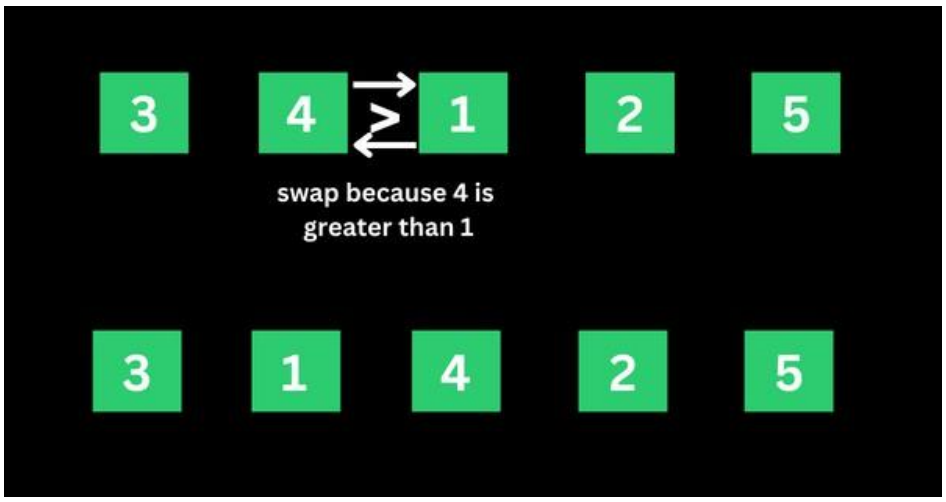
That's the first iteration. And the numbers are now arranged as 3, 4, 1, 2, and 5 – from the initial 5, 3, 4, 1, and 2. As you might realize, 5 should be the last number if the numbers are sorted in ascending order. This means the first iteration is really completed.

Second Iteration of the Sorting and the Rest

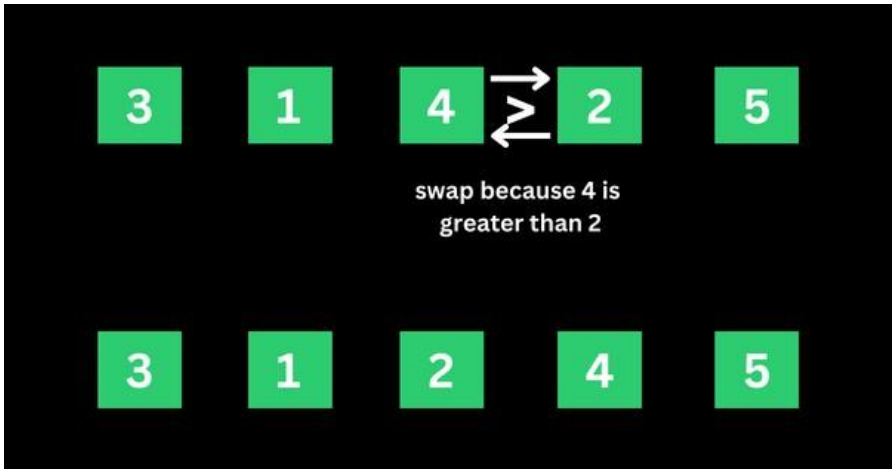
The algorithm starts the second iteration with the last result of 3, 4, 1, 2, and 5. This time around, 3 is smaller than 4, so no swapping happens. This means the numbers will remain the same.



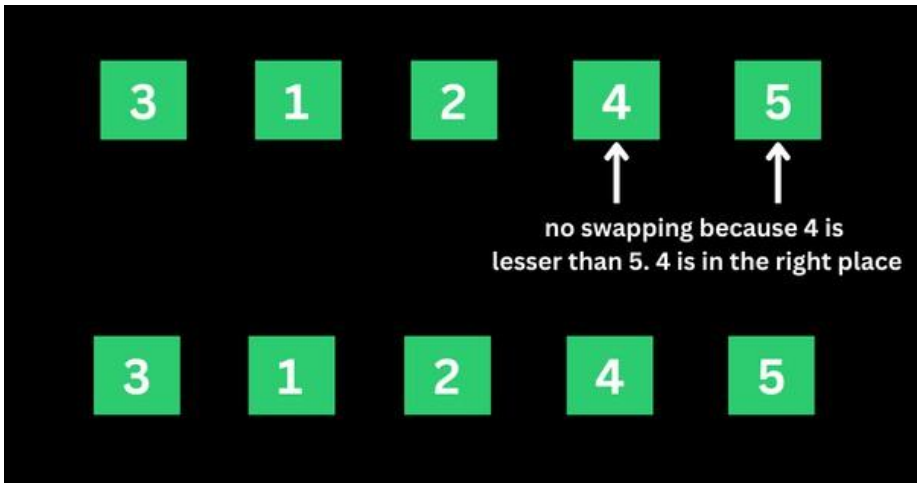
The algorithm proceeds to compare 4 and 1. 4 is greater than 1, so 4 is swapped for 1 and the numbers become 3, 1, 4, 2, and 5.



The algorithm now proceeds to compare 4 and 2. 4 is greater than 2, so 4 is swapped for 2 and the numbers become 3, 1, 2, 4, and 5.



4 is now in the right place, so no swapping occurs between 4 and 5 because 4 is smaller than 5.



That's how the algorithm continues to compare the numbers until they are arranged in ascending order of 1, 2, 3, 4, and 5.

Concept of OpenMP

- OpenMP (Open Multi-Processing) is an application programming interface (API) that supports shared-memory parallel programming in C, C++, and Fortran. It is used to write parallel programs that can run on multicore processors, multiprocessor systems, and parallel computing clusters.
- OpenMP provides a set of directives and functions that can be inserted into the source code of a program to parallelize its execution. These directives are simple and easy to use, and they can be applied to loops, sections, functions, and other program constructs. The compiler then generates parallel code that can run on multiple processors concurrently.
- OpenMP programs are designed to take advantage of the shared-memory architecture of modern

processors, where multiple processor cores can access the same memory. OpenMP uses a fork-join model of parallel execution, where a master thread forks multiple worker threads to execute a parallel region of the code, and then waits for all threads to complete before continuing with the sequential part of the code.

How Parallel Bubble Sort Work

- Parallel Bubble Sort is a modification of the classic Bubble Sort algorithm that takes advantage of parallel processing to speed up the sorting process.
- In parallel Bubble Sort, the list of elements is divided into multiple sublists that are sorted concurrently by multiple threads. Each thread sorts its sublist using the regular Bubble Sort algorithm. When all sublists have been sorted, they are merged together to form the final sorted list.
- The parallelization of the algorithm is achieved using OpenMP, a programming API that supports parallel processing in C++, Fortran, and other programming languages. OpenMP provides a set of compiler directives that allow developers to specify which parts of the code can be executed in parallel.
- In the parallel Bubble Sort algorithm, the main loop that iterates over the list of elements is divided into multiple iterations that are executed concurrently by multiple threads. Each thread sorts a subset of the list, and the threads synchronize their work at the end of each iteration to ensure that the elements are properly ordered.
- Parallel Bubble Sort can provide a significant speedup over the regular Bubble Sort algorithm, especially when sorting large datasets on multi-core processors. However, the speedup is limited by the overhead of thread creation and synchronization, and it may not be worth the effort for small datasets or when using a single-core processor.

How to measure the performance of sequential and parallel algorithms?

To measure the performance of sequential Bubble sort and parallel Bubble sort algorithms, you can follow these steps:

1. Implement both the sequential and parallel Bubble sort algorithms.
2. Choose a range of test cases, such as arrays of different sizes and different degrees of sortedness, to test the performance of both algorithms.

3. Use a reliable timer to measure the execution time of each algorithm on each test case.
4. Record the execution times and analyze the results.

When measuring the performance of the parallel Bubble sort algorithm, you will need to specify the number of threads to use. You can experiment with different numbers of threads to find the optimal value for your system.

Here are some additional tips for measuring performance:

- Run each algorithm multiple times on each test case and take the average execution time to reduce the impact of variations in system load and other factors.
- Monitor system resource usage during execution, such as CPU utilization and memory consumption, to detect any performance bottlenecks.
- Visualize the results using charts or graphs to make it easier to compare the performance of the two algorithms.

How to check CPU utilization and memory consumption in ubuntu

In Ubuntu, you can use a variety of tools to check CPU utilization and memory consumption. Here are some common tools:

1. **top:** The top command provides a real-time view of system resource usage, including CPU utilization and memory consumption. To use it, open a terminal window and type top. The output will display a list of processes sorted by resource usage, with the most resource-intensive processes at the top.
2. **htop**: htop is a more advanced version of top that provides additional features, such as interactive process filtering and a color-coded display. To use it, open a terminal window and type htop.
3. **ps**: The ps command provides a snapshot of system resource usage at a particular moment in time. To use it, open a terminal window and type ps aux. This will display a list of all running processes and their resource usage.
4. **free:** The free command provides information about system memory usage, including total, used, and free memory. To use it, open a terminal window and type free -h.
5. **vmstat:** The vmstat command provides a variety of system statistics, including CPU utilization, memory usage, and disk activity. To use it, open a terminal window and type vmstat.

Conclusion- In this way we can implement Bubble Sort in parallel way using OpenMP also come to know how to how to measure performance of serial and parallel algorithm

Assignment Question

1. **What is parallel Bubble Sort?**
2. **How does Parallel Bubble Sort work?**
3. **How do you implement Parallel Bubble Sort using OpenMP?**
4. **What are the advantages of Parallel Bubble Sort?**
5. **Difference between serial bubble sort and parallel bubble sort**

Reference link

- https://www.freecodecamp.org/news/bubble-sort-algorithm-in-java-cpp-python-with-example-code/

Assignment 2(a)

Title: Write a program to implement Parallel Bubble Sort using OpenMP

```
#include<iostream>
#include<stdlib.h>
#include<omp.h>
using namespace std;

void bubble(int *, int);
void swap(int &, int &);


void bubble(int *a, int n)
{
   for(  int i = 0;  i < n;  i++ )
    {
          int first = i % 2;

          #pragma omp parallel for shared(a,first)
          for(  int j = first;  j < n-1;  j += 2  )
           {
                  if(  a[ j ]  >  a[ j+1 ]  )
                   {
                          swap(  a[ j ],  a[ j+1 ]  );
                   }
                    }
    }
}


void swap(int &a, int &b)
{

   int test;
   test=a;
   a=b;
   b=test;

}

int main()
{
```

```
    int *a,n;
    cout<<"\n enter total no of elements=>";
    cin>>n;
    a=new int[n];
    cout<<"\n enter elements=>";
    for(int i=0;i<n;i++)
    {
            cin>>a[i];
    }

    bubble(a,n);

    cout<<"\n sorted array is=>";
    for(int i=0;i<n;i++)
    {
            cout<<a[i]<<endl;
    }


return 0;
}
```

Here, we're using the #pragma omp parallel directive to parallelize the code using OpenMP. Within the parallel block, we have a while loop that runs until the array is sorted. Inside the while loop, we have a #pragma omp for directive that parallelizes the for loop that performs the bubble sort.

Note that the flag variable is used to keep track of whether any swaps were made during an iteration of the for loop. If no swaps were made, then the array is already sorted and we can exit the loop.

explanation of each line of the code:

These lines are the standard C++ header files used for input/output operations and OpenMP, respectively.

```c++
void parallel_bubble_sort(int arr[], int n) {
    #pragma omp parallel
    {
        int i, temp, flag = 1;
        while (flag) {
            flag = 0;
            #pragma omp for
            for (i = 0; i < n-1; i++) {
                if (arr[i] > arr[i+1]) {
                    temp = arr[i];
                    arr[i] = arr[i+1];
                    arr[i+1] = temp;
                    flag = 1;
                }
            }
        }
    }
}
```

This is the function parallel_bubble_sort() that implements the parallel bubble sort algorithm using OpenMP.

- #pragma omp parallel is a directive that creates a team of threads to execute the parallel code inside the block. In this case, the block contains the code for bubble sort algorithm.
- int i, temp, flag = 1; declares the variables i, temp, and flag that will be used inside the while loop.
- while (flag) is a loop that runs until the flag variable is 0.
- flag = 0; sets the flag variable to 0 before starting each iteration of the for loop.
- #pragma omp for is a directive that parallelizes the for loop, by dividing the loop iterations among the threads in the team. Each thread performs the sorting operation on a subset of the array, thereby making the sorting process faster.
- for (i = 0; i < n-1; i++) is a for loop that iterates over the array, from 0 to n-1.
- if (arr[i] > arr[i+1]) checks if the current element is greater than the next element.
- temp = arr[i]; arr[i] = arr[i+1]; arr[i+1] = temp; swaps the current element with the next element, using a temporary variable.
- flag = 1; sets the flag variable to 1, indicating that a swap has been made.
- Finally, the sorted array is printed using a for loop.

```c++
int main() {
    int arr[] = {5, 3, 1, 9, 8, 2, 4, 7, 6};
    int n = sizeof(arr)/sizeof(arr[0]);

    parallel_bubble_sort(arr, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

This is the main() function, which initializes an array arr and its size n. The function parallel_bubble_sort() is called with these arguments to sort the array. The sorted array is then printed to the console using a for loop.

How to Run code in Ubuntu:

1. **Open a terminal window.**

Compile the code using the following command:

```
g++ -fopenmp filename.cpp -o executable
```

Run the program by executing the following command:

```bash
./executable
```

Output

```c
Sorted array: 1 2 3 4 5 6 7 8 9
```

This is because the input array `{5, 3, 1, 9, 8, 2, 4, 7, 6}` is sorted in ascending order using the parallel bubble sort algorithm implemented in the `parallel_bubble_sort()` function. The sorted array is then printed to the console in the `main()` function using a for loop.

To measure the performance of sequential Bubble sort and parallel Bubble sort algorithms, you can follow these steps:

1. Implement both the sequential and parallel Bubble sort algorithms.
2. Choose a range of test cases, such as arrays of different sizes and different degrees of sortedness, to test the performance of both algorithms.
3. Use a reliable timer to measure the execution time of each algorithm on each test case.
4. Record the execution times and analyze the results.

When measuring the performance of the parallel Bubble sort algorithm, you will need to specify the number of threads to use. You can experiment with different numbers of threads to find the optimal value for your system.

Here are some additional tips for measuring performance:

- Run each algorithm multiple times on each test case and take the average execution time to reduce the impact of variations in system load and other factors.
- Monitor system resource usage during execution, such as CPU utilization and memory consumption, to detect any performance bottlenecks.
- Visualize the results using charts or graphs to make it easier to compare the performance of the two algorithms.

How to check CPU utilisation and memory consumption in ubuntu

In Ubuntu, you can use a variety of tools to check CPU utilization and memory consumption. Here are some common tools:

1. top: The top command provides a real-time view of system resource usage, including CPU utilization and memory consumption. To use it, open a terminal window and type top. The output will display a list of processes sorted by resource usage, with the most resource-intensive processes at the top.
2. htop: htop is a more advanced version of top that provides additional features, such as interactive process filtering and a color-coded display. To use it, open a terminal window and type htop.
3. ps: The ps command provides a snapshot of system resource usage at a particular moment in time. To use it, open a terminal window and type ps aux. This will display a list of all running processes and their resource usage.

4. free: The free command provides information about system memory usage, including total, used, and free memory. To use it, open a terminal window and type free -h.
5. vmstat: The vmstat command provides a variety of system statistics, including CPU utilization, memory usage, and disk activity. To use it, open a terminal window and type vmstat.

Group A

Assignment No: 2(B)

Title of the Assignment: Write a program to implement Parallel Merge Sort. Use existing algorithms and measure the performance of sequential and parallel algorithms.

Objective of the Assignment: Students should be able to Write a program to implement Parallel Merge Sort and can measure the performance of sequential and parallel algorithms.

Prerequisite:

1. Basic of programming language
2. Concept of Merge Sort
3. Concept of Parallelism

Contents for Theory:

1. **What is Merge? Use of Merge Sort**
2. **Example of Merge sort?**
3. **Concept of OpenMP**
4. **How Parallel Merge Sort Work**
5. **How to measure the performance of sequential and parallel algorithms?**

What is Merge Sort?

Merge sort is a sorting algorithm that uses a divide-and-conquer approach to sort an array or a list of elements. The algorithm works by recursively dividing the input array into two halves, sorting each half, and then merging the sorted halves to produce a sorted output.

The merge sort algorithm can be broken down into the following steps:

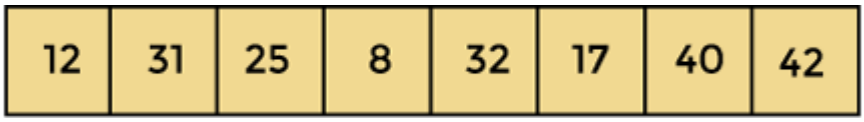
1. Divide the input array into two halves.
2. Recursively sort the left half of the array.
3. Recursively sort the right half of the array.
4. Merge the two sorted halves into a single sorted output array.

- The merging step is where the bulk of the work happens in merge sort. The algorithm compares the first elements of each sorted half, selects the smaller element, and appends it to the output array. This process continues until all elements from both halves have been appended to the output array.
- The time complexity of merge sort is O(n log n), which makes it an efficient sorting algorithm for large input arrays. However, merge sort also requires additional memory to store the output array, which can make it less suitable for use with limited memory resources.
- In simple terms, we can say that the process of merge sort is to divide the array into two halves, sort each half, and then merge the sorted halves back together. This process is repeated until the entire array is sorted.
- One thing that you might wonder is what is the specialty of this algorithm. We already have a number of sorting algorithms then why do we need this algorithm? One of the main advantages of merge sort is that it has a time complexity of O(n log n), which means it can sort large arrays relatively quickly. It is also a stable sort, which means that the order of elements with equal values is preserved during the sort.
- Merge sort is a popular choice for sorting large datasets because it is relatively efficient and easy to implement. It is often used in conjunction with other algorithms, such as quicksort, to improve the overall performance of a sorting routine.

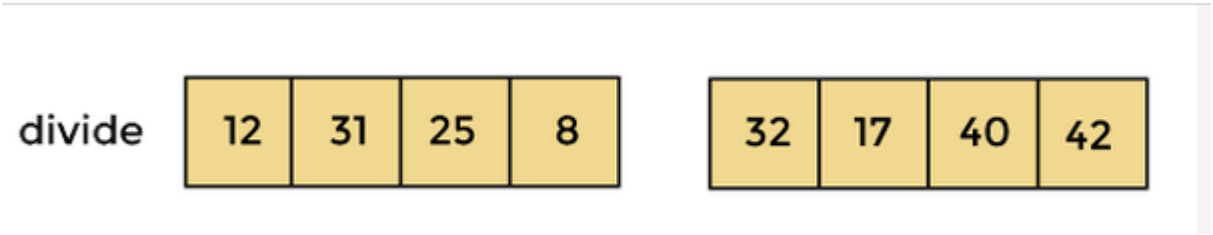
Example of Merge sort

Now, let's see the working of merge sort Algorithm. To understand the working of the merge sort algorithm, let's take an unsorted array. It will be easier to understand the merge sort via an example.

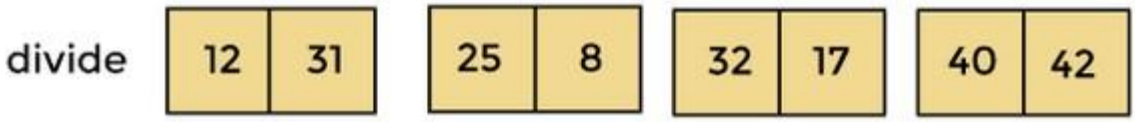
Let the elements of array are -



- According to the merge sort, first divide the given array into two equal halves. Merge sort keeps dividing the list into equal parts until it cannot be further divided.
- As there are eight elements in the given array, so it is divided into two arrays of size 4.



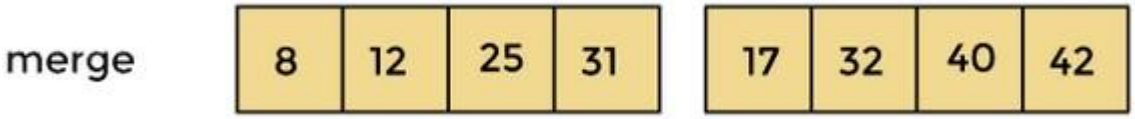
- Now, again divide these two arrays into halves. As they are of size 4, divide them into new arrays of size 2.



- Now, again divide these arrays to get the atomic value that cannot be further divided.

- Now, combine them in the same manner they were broken.
- In combining, first compare the element of each array and then combine them into another array in sorted order.
- So, first compare 12 and 31, both are in sorted positions. Then compare 25 and 8, and in the list of two values, put 8 first followed by 25. Then compare 32 and 17, sort them and put 17 first followed by 32. After that, compare 40 and 42, and place them sequentially.

- In the next iteration of combining, now compare the arrays with two data values and merge them into an array of found values in sorted order.



- Now, there is a final merging of the arrays. After the final merging of above arrays, the array will look like -

Concept of OpenMP

- OpenMP (Open Multi-Processing) is an application programming interface (API) that supports shared-memory parallel programming in C, C++, and Fortran. It is used to write parallel programs that can run on multicore processors, multiprocessor systems, and parallel computing clusters.
- OpenMP provides a set of directives and functions that can be inserted into the source code of a program to parallelize its execution. These directives are simple and easy to use, and they can be applied to loops, sections, functions, and other program constructs. The compiler then generates parallel code that can run on multiple processors concurrently.
- OpenMP programs are designed to take advantage of the shared-memory architecture of modern processors, where multiple processor cores can access the same memory. OpenMP uses a fork-join model of parallel execution, where a master thread forks multiple worker threads to execute a parallel region of the code, and then waits for all threads to complete before continuing with the sequential part of the code.

How Parallel Merge Sort Work

- Parallel merge sort is a parallelized version of the merge sort algorithm that takes advantage of multiple processors or cores to improve its performance. In parallel merge sort, the input array is divided into smaller subarrays, which are sorted in parallel using multiple processors or cores. The sorted subarrays are then merged together in parallel to produce the final sorted output.
- The parallel merge sort algorithm can be broken down into the following steps:

- Divide the input array into smaller subarrays.
- Assign each subarray to a separate processor or core for sorting.
- Sort each subarray in parallel using the merge sort algorithm.
- Merge the sorted subarrays together in parallel to produce the final sorted output.
- The merging step in parallel merge sort is performed in a similar way to the merging step in the sequential merge sort algorithm. However, because the subarrays are sorted in parallel, the merging step can also be performed in parallel using multiple processors or cores. This can significantly reduce the time required to merge the sorted subarrays and produce the final output.
- Parallel merge sort can provide significant performance benefits for large input arrays with many elements, especially when running on hardware with multiple processors or cores. However, it also requires additional overhead to manage the parallelization, and may not always provide performance improvements for smaller input sizes or when run on hardware with limited parallel processing capabilities.

How to measure the performance of sequential and parallel algorithms?

There are several metrics that can be used to measure the performance of sequential and parallel merge sort algorithms:

1. **Execution time:** Execution time is the amount of time it takes for the algorithm to complete its sorting operation. This metric can be used to compare the speed of sequential and parallel merge sort algorithms.
2. **Speedup**: Speedup is the ratio of the execution time of the sequential merge sort algorithm to the execution time of the parallel merge sort algorithm. A speedup of greater than 1 indicates that the parallel algorithm is faster than the sequential algorithm.
3. **Efficiency:** Efficiency is the ratio of the speedup to the number of processors or cores used in the parallel algorithm. This metric can be used to determine how well the parallel algorithm is utilizing the available resources.
4. **Scalability**: Scalability is the ability of the algorithm to maintain its performance as the input size and number of processors or cores increase. A scalable algorithm will maintain a consistent speedup and efficiency as more resources are added.

To measure the performance of sequential and parallel merge sort algorithms, you can perform experiments on different input sizes and numbers of processors or cores. By measuring the execution time, speedup, efficiency, and scalability of the algorithms under different conditions, you can determine

which algorithm is more efficient for different input sizes and hardware configurations. Additionally, you can use profiling tools to analyze the performance of the algorithms and identify areas for optimization

Conclusion- In this way we can implement Merge Sort in parallel way using OpenMP also come to know how to how to measure performance of serial and parallel algorithm

Assignment Question

1. **What is parallel Merge Sort?**
2. **How does Parallel Merge Sort work?**
3. **How do you implement Parallel MergeSort using OpenMP?**
4. **What are the advantages of Parallel MergeSort?**
5. **Difference between serial Mergesort and parallel Mergesort**

Reference link

- https://www.geeksforgeeks.org/merge-sort/
- https://www.javatpoint.com/merge-sort

```
#include<iostream>
#include<stdlib.h>
#include<omp.h>
using namespace std;


void mergesort(int a[],int i,int j);
void merge(int a[],int i1,int j1,int i2,int j2);

void mergesort(int a[],int i,int j)
{
	int mid;
	if(i<j)
	{
	mid=(i+j)/2;

	#pragma omp parallel sections
	{

	#pragma omp section
	{
		mergesort(a,i,mid);
	}

	#pragma omp section
	{
		mergesort(a,mid+1,j);
	}
	}

	merge(a,i,mid,mid+1,j);
	}

}

void merge(int a[],int i1,int j1,int i2,int j2)
{
	int temp[1000];
```

```
        int i,j,k;
        i=i1;
        j=i2;
        k=0;

        while(i<=j1 && j<=j2)
        {
        if(a[i]<a[j])
        {
        temp[k++]=a[i++];
        }
        else
        {
        temp[k++]=a[j++];
   }
        }

        while(i<=j1)
        {
        temp[k++]=a[i++];
        }

        while(j<=j2)
        {
        temp[k++]=a[j++];
        }

        for(i=i1,j=0;i<=j2;i++,j++)
        {
        a[i]=temp[j];
        }
}


int main()
{
        int *a,n,i;
        cout<<"\n enter total no of elements=>";
        cin>>n;
        a= new int[n];
```

```
        cout<<"\n enter elements=>";
        for(i=0;i<n;i++)
        {
        cin>>a[i];
        }
   //     start=.....
//#pragma omp…..
        mergesort(a, 0, n-1);
//          stop…….
        cout<<"\n sorted array is=>";
        for(i=0;i<n;i++)
        {
        cout<<"\n"<<a[i];
        }
        // Cout<<Stop-Start
        return 0;
}
```

Second Code:

```
#include <iostream>
#include <omp.h>

void merge(int* arr, int l, int m, int r) {
        int i, j, k;
        int n1 = m - l + 1;
        int n2 = r - m;

        int L[n1], R[n2];

        for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
        for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

        i = 0;
        j = 0;
        k = l;
        while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
        }
```

```
        else {
        arr[k] = R[j];
        j++;
        }
        k++;
        }

        while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
        }

        while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
        }
}

void mergeSort(int* arr, int l, int r) {
        if (l < r) {
        int m = l + (r - l) / 2;
        #pragma omp parallel sections
        {
        #pragma omp section
        {
                mergeSort(arr, l, m);
        }
        #pragma omp section
        {
                mergeSort(arr, m + 1, r);
        }
        }

        merge(arr, l, m, r);
        }
}

int main() {
        int arr[] = { 12, 11, 13, 5, 6, 7 };
        int n = sizeof(arr) / sizeof(arr[0]);
  double start, stop;
        std::cout << "Given array is: ";
        for (int i = 0; i < n; i++)
        std::cout << arr[i] << " ";
        std::cout << std::endl;
```

```
 start = omp_get_wtime();
  #pragma omp parallel
  {
        mergeSort(arr, 0, n - 1);
}
  stop = omp_get_wtime();
        std::cout << "Sorted array is: ";
        for (int i = 0; i < n; i++)
        std::cout << arr[i] << " ";
        std::cout << std::endl;

std::cout<<stop-start;
        return 0;
}
```

Explanation

```c++
#include<iostream>
#include<stdlib.h>
#include<omp.h>
using namespace std;
```

This block of code is a set of header files and a namespace declaration. iostream and stdlib.h are header files, and omp.h is the header file for OpenMP (Open Multi-Processing) library that is used for parallel programming.

```c++
void mergesort(int a[],int i,int j);
void merge(int a[],int i1,int j1,int i2,int j2);
```

These are function declarations for mergesort and merge functions.

```c
void mergesort(int a[],int i,int j)
{
    int mid;
    if(i<j)
    {
        mid=(i+j)/2;

        #pragma omp parallel sections
        {
            #pragma omp section
            {
                mergesort(a,i,mid);
            }

            #pragma omp section
            {
                mergesort(a,mid+1,j);
            }
        }

        merge(a,i,mid,mid+1,j);
    }

}
```

This is the mergesort function, which implements the merge sort algorithm recursively. a[] is the input array that is to be sorted, i and j are the starting and ending indices of the subarray being sorted.

First, it checks whether i is less than j because if i is greater than or equal to j, it means that there is only one element in the subarray, which is already sorted. Then it calculates the midpoint of the subarray using (i+j)/2.

The #pragma omp parallel sections directive creates a parallel region that specifies two parallel sections that can be executed concurrently. The mergesort function is then recursively called on the left and right halves of the subarray in two separate parallel sections.

Finally, the merge function is called to merge the sorted halves of the subarray.

c Copy code

```c
void merge(int a[],int i1,int j1,int i2,int j2)
```

This is the function signature which takes an integer array a and four integer variables i1, j1, i2, and j2. i1 and j1 define the start and end indices of the first sorted subarray, and i2 and j2 define the start and end indices of the second sorted subarray.

```c
int temp[1000];
int i,j,k;
i=i1;
j=i2;
k=0;
```

Here, a temporary array temp is created with a size of 1000. Three integer variables i, j, and k are initialized. i and j are set to the start indices of the two subarrays, while k is set to 0.

```c
while(i<=j1 && j<=j2)
{
    if(a[i]<a[j])
    {
        temp[k++]=a[i++];
    }
    else
    {
        temp[k++]=a[j++];
    }
}
```

This is a while loop that runs as long as i is less than or equal to j1 and j is less than or equal to j2. Inside the loop, if the element at index i of the first subarray is less than the element at index j of the second subarray, then the element at index i is copied to the temp array at index k, and i and k are incremented. Otherwise, the element at index j is copied to the temp array at index k, and j and k are incremented.

```
while(i<=j1)
{
    temp[k++]=a[i++];
}

while(j<=j2)
{
    temp[k++]=a[j++];
}
```

After the above loop terminates, there may be some elements left in one of the subarrays. These loops copy the remaining elements into the temp array.

```
for(i=i1,j=0;i<=j2;i++,j++)
{
    a[i]=temp[j];
}
```

Finally, the sorted temp array is copied back to the original a array. The loop runs from i1 to j2 and copies the elements of temp array to the corresponding indices in the a array. The loop variable j starts from 0 and increments alongside i.

Group A

Assignment No: 3

Title of the Assignment: Implement Min, Max, Sum and Average operations using Parallel Reduction.

Objective of the Assignment: To understand the concept of parallel reduction and how it can be used to perform basic mathematical operations on given data sets.

Prerequisite:

1. Parallel computing architectures
2. Parallel programming models
3. Proficiency in programming languages

Contents for Theory:

1. What is parallel reduction and its usefulness for mathematical operations on large data?
2. Concept of OpenMP
3. How do parallel reduction algorithms for Min, Max, Sum, and Average work, and what are their advantages and limitations?

Parallel Reduction.

Here's a **function-wise manual** on how to understand and run the sample C++ program that demonstrates how to implement Min, Max, Sum, and Average operations using parallel reduction.

1. **Min_Reduction function**
    - The function takes in a vector of integers as input and finds the minimum value in the vector using parallel reduction.
    - The OpenMP reduction clause is used with the "min" operator to find the minimum value across all threads.
    - The minimum value found by each thread is reduced to the overall minimum value of the entire array.
    - The final minimum value is printed to the console.

2. **Max_Reduction function**
    - The function takes in a vector of integers as input and finds the maximum value in the vector using parallel reduction.
    - The OpenMP reduction clause is used with the "max" operator to find the maximum value across all threads.
    - The maximum value found by each thread is reduced to the overall maximum value of the entire array.
    - The final maximum value is printed to the console.

3. **Sum_Reduction function**
    - The function takes in a vector of integers as input and finds the sum of all the values in the vector using parallel reduction.
    - The OpenMP reduction clause is used with the "+" operator to find the sum across all threads.
    - The sum found by each thread is reduced to the overall sum of the entire array.
    - The final sum is printed to the console.

4. **Average_Reduction function**
    - The function takes in a vector of integers as input and finds the average of all the values in the vector using parallel reduction.
    - The OpenMP reduction clause is used with the "+" operator to find the sum across all threads.

- The sum found by each thread is reduced to the overall sum of the entire array.
- The final sum is divided by the size of the array to find the average.
- The final average value is printed to the console.

5. **Main Function**
   - The function initializes a vector of integers with some values.
   - The function calls the min_reduction, max_reduction, sum_reduction, and average_reduction functions on the input vector to find the corresponding values.
   - The final minimum, maximum, sum, and average values are printed to the console.
6. **Compiling and running the program**

Compile the program: You need to use a C++ compiler that supports OpenMP, such as g++ or clang. Open a terminal and navigate to the directory where your program is saved. Then, compile the program using the following command:

```
$ g++ -fopenmp program.cpp -o program
```

This command compiles your program and creates an executable file named "program". The "-fopenmp" flag tells the compiler to enable OpenMP.

Run the program: To run the program, simply type the name of the executable file in the terminal and press Enter:

```
$ ./program
```

Conclusion: We have implemented the Min, Max, Sum, and Average operations using parallel reduction in C++ with OpenMP. Parallel reduction is a powerful technique that allows us to perform these operations on large arrays more efficiently by dividing the work among multiple threads running in parallel. We presented a code example that demonstrates the implementation of these operations using parallel reduction in C++ with OpenMP. We also provided a manual for running OpenMP programs on the Ubuntu platform.

Assignment Question

1. What are the benefits of using parallel reduction for basic operations on large arrays?
2. How does OpenMP's "reduction" clause work in parallel reduction?
3. How do you set up a C++ program for parallel computation with OpenMP?
4. What are the performance characteristics of parallel reduction, and how do they vary based on input size?
5. How can you modify the provided code example for more complex operations using parallel reduction?

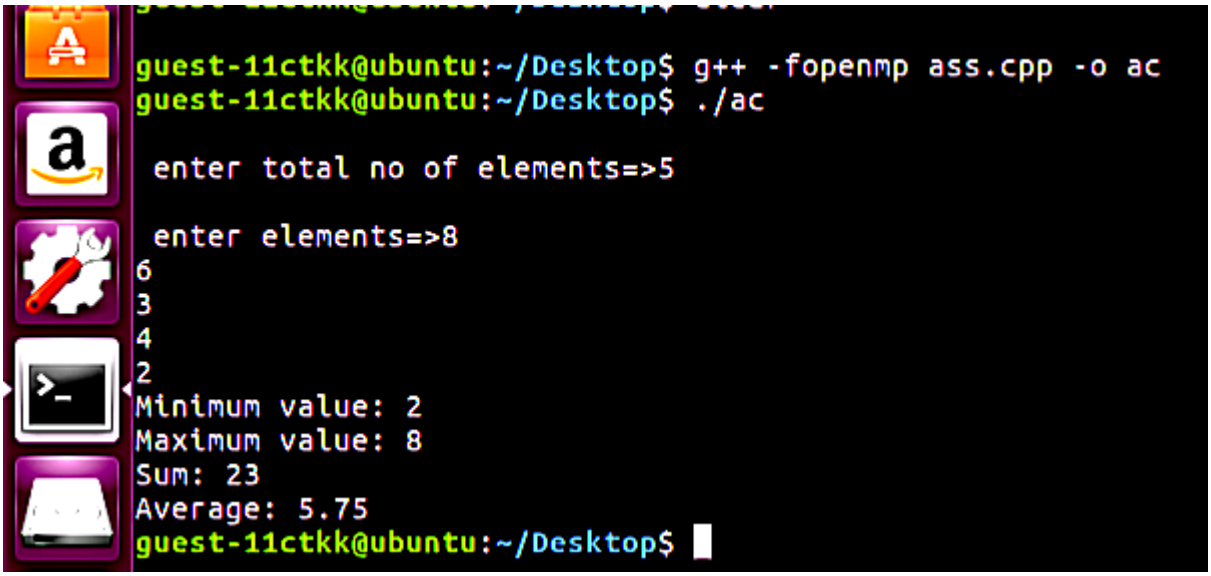
Grp A_Assignment 3

Implement Min, Max, Sum and Average operations using Parallel Reduction.

```
#include <iostream>
//#include <vector>
#include <omp.h>
#include <climits>
using namespace std;
void min_reduction(int arr[], int n) {
  int min_value = INT_MAX;
  #pragma omp parallel for reduction(min: min_value)
  for (int i = 0; i < n; i++) {
            if (arr[i] < min_value) {
            min_value = arr[i];
            }
  }
  cout << "Minimum value: " << min_value << endl;
}

void max_reduction(int arr[], int n) {
  int max_value = INT_MIN;
  #pragma omp parallel for reduction(max: max_value)
  for (int i = 0; i < n; i++) {
            if (arr[i] > max_value) {
            max_value = arr[i];
            }
  }
  cout << "Maximum value: " << max_value << endl;
}

void sum_reduction(int arr[], int n) {
  int sum = 0;
   #pragma omp parallel for reduction(+: sum)
   for (int i = 0; i < n; i++) {
            sum += arr[i];
  }
  cout << "Sum: " << sum << endl;
}

void average_reduction(int arr[], int n) {
  int sum = 0;
```

```
 #pragma omp parallel for reduction(+: sum)
 for (int i = 0; i < n; i++) {
        sum += arr[i];
 }
 cout << "Average: " << (double)sum / (n-1) << endl;
}

int main() {
   int *arr,n;
   cout<<"\n enter total no of elements=>";
   cin>>n;
   arr=new int[n];
   cout<<"\n enter elements=>";
   for(int i=0;i<n;i++)
   {
        cin>>arr[i];
   }

//   int arr[] = {5, 2, 9, 1, 7, 6, 8, 3, 4};
//   int n = size(arr);

 min_reduction(arr, n);
 max_reduction(arr, n);
 sum_reduction(arr, n);
 average_reduction(arr, n);
}
```

Output



Void Min_reduction()

- void min_reduction(vector<int>& arr) declares a void function that takes a reference to an integer vector as its argument.
- int min_value = INT_MAX; initializes an integer variable min_value to the largest possible integer value using the INT_MAX constant from the <climits> header file. This is done to ensure that min_value is initially greater than any element in arr.

- #pragma omp parallel for reduction(min: min_value) is an OpenMP directive that specifies that the following loop should be executed in parallel using multiple threads. The reduction(min: min_value) clause indicates that each thread should maintain a private copy of min_value and update it with the minimum value it finds in its portion of the loop. Once the loop is complete, OpenMP will combine all the private copies of min_value into a single shared value that represents the minimum value in arr.
- for (int i = 0; i < arr.size(); i++) { is a loop that iterates over each element of arr.
- if (arr[i] < min_value) { min_value = arr[i]; } checks if the current element of arr is less than min_value. If so, it updates min_value to be the current element.
- cout << "Minimum value: " << min_value << endl; prints out the minimum value found in arr.

void max_reduction()

- void max_reduction(vector<int>& arr) declares a void function that takes a reference to an integer vector as its argument.
- int max_value = INT_MIN; initializes an integer variable max_value to the smallest possible integer value using the INT_MIN constant from the <climits> header file. This is done to ensure that max_value is initially smaller than any element in arr.
- #pragma omp parallel for reduction(max: max_value) is an OpenMP directive that specifies that the following loop should be executed in parallel using multiple threads. The reduction(max: max_value) clause indicates that each thread should maintain a private copy of max_value and update it with the maximum value it finds in its portion of the loop. Once the loop is complete, OpenMP will combine all the private copies of max_value into a single shared value that represents the maximum value in arr.
- for (int i = 0; i < arr.size(); i++) { is a loop that iterates over each element of arr.
- if (arr[i] > max_value) { max_value = arr[i]; } checks if the current element of arr is greater than max_value. If so, it updates max_value to be the current element.
- cout << "Maximum value: " << max_value << endl; prints out the maximum value found in arr.

#include <climits>

<climits> is a header file in C++ that contains constants related to integer types. This header file provides implementation-defined constants for minimum and maximum values of integral types, such as INT_MAX (maximum value of int) and INT_MIN (minimum value of int).

Using these constants instead of hardcoding the values of the minimum and maximum integer values is a good practice because it makes the code more readable and avoids the possibility of introducing errors in the code. The use of these constants also ensures that the code will work correctly across different platforms and compilers.

INT_MIN :

Minimum value for an object of type int

Value of INT_MIN is -32767 ($-2^{15}+1$) or less*

INT_MAX :

Maximum value for an object of type int

Value of INT_MAX is 2147483647 (-2^{31} to $2^{31}-1$)

Group A

Assignment 4(A)

Title of the Assignment: Write a CUDA Program for Addition of two large vectors

Objective of the Assignment: Students should be able to perform CUDA Program for Addition of two large vectors

Prerequisite:

1. CUDA Concept
2. Vector Addition
3. How to execute Program in CUDA Environment

Contents for Theory:

1. **What is CUDA**
2. **Addition of two large Vector**
3. **Execution of CUDA Environment**

What is CUDA

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA. It allows developers to use the power of NVIDIA graphics processing units (GPUs) to accelerate computation tasks in various applications, including scientific computing, machine learning, and computer vision.CUDA provides a set of programming APIs, libraries, and tools that enable developers to write and execute parallel code on NVIDIA GPUs. It supports popular programming languages like C, C++, and Python, and provides a simple programming model that abstracts away much of the low-level details of GPU architecture.

Using CUDA, developers can exploit the massive parallelism and high computational power of GPUs to accelerate computationally intensive tasks, such as matrix operations, image processing, and deep learning. CUDA has become an important tool for scientific research and is widely used in fields like physics, chemistry, biology, and engineering.

Steps for Addition of two large vectors using CUDA

1. Define the size of the vectors: In this step, you need to define the size of the vectors that you want to add. This will determine the number of threads and blocks you will need to use to parallelize the addition operation.
2. Allocate memory on the host: In this step, you need to allocate memory on the host for the two vectors that you want to add and for the result vector. You can use the C malloc function to allocate memory.
3. Initialize the vectors: In this step, you need to initialize the two vectors that you want to add on the host. You can use a loop to fill the vectors with data.
4. Allocate memory on the device: In this step, you need to allocate memory on the device for the two vectors that you want to add and for the result vector. You can use the CUDA function cudaMalloc to allocate memory.
5. Copy the input vectors from host to device: In this step, you need to copy the two input vectors from the host to the device memory. You can use the CUDA function cudaMemcpy to copy the vectors.
6. Launch the kernel: In this step, you need to launch the CUDA kernel that will perform the addition operation. The kernel will be executed by multiple threads in parallel. You can use the <<<...>>> syntax to specify the number of blocks and threads to use.
7. Copy the result vector from device to host: In this step, you need to copy the result vector from the device memory to the host memory. You can use the CUDA function cudaMemcpy to copy the result vector.

8. Free memory on the device: In this step, you need to free the memory that was allocated on the device. You can use the CUDA function cudaFree to free the memory.
9. Free memory on the host: In this step, you need to free the memory that was allocated on the host. You can use the C free function to free the memory.

Execution of Program over CUDA Environment

Here are the steps to run a CUDA program for adding two large vectors:

1. Install CUDA Toolkit: First, you need to install the CUDA Toolkit on your system. You can download the CUDA Toolkit from the NVIDIA website and follow the installation instructions provided.
2. Set up CUDA environment: Once the CUDA Toolkit is installed, you need to set up the CUDA environment on your system. This involves setting the PATH and LD_LIBRARY_PATH environment variables to the appropriate directories.
3. Write the CUDA program: You need to write a CUDA program that performs the addition of two large vectors. You can use a text editor to write the program and save it with a .cu extension.
4. Compile the CUDA program: You need to compile the CUDA program using the nvcc compiler that comes with the CUDA Toolkit. The command to compile the program is:

```
nvcc -o program_name program_name.cu
```

5. This will generate an executable program named program_name.

Run the CUDA program: Finally, you can run the CUDA program by executing the executable file generated in the previous step. The command to run the program is:

```
./program_name
```

This will execute the program and perform the addition of two large vectors.

Questions:

1. What is the purpose of using CUDA to perform addition of two large vectors?
2. How do you allocate memory for the vectors on the device using CUDA?
3. How do you launch the CUDA kernel to perform the addition of two large vectors?
4. How can you optimize the performance of the CUDA program for adding two large vectors

```
#include <iostream>
#include <cuda_runtime.h>
#include /usr/local/cuda/include/cuda_runtime.h

_global_void addVectors(int* A, int* B, int* C, int n)
{
        int i = blockIdx.x * blockDim.x + threadIdx.x;
        if (i < n)
        {
        C[i] = A[i] + B[i];
        }
}

int main()
{
        int n = 1000000;
        int* A, * B, * C;
        int size = n * sizeof(int);

        // Allocate memory on the host
        cudaMallocHost(&A, size);
        cudaMallocHost(&B, size);
        cudaMallocHost(&C, size);

        // Initialize the vectors
        for (int i = 0; i < n; i++)
        {
        A[i] = i;
        B[i] = i * 2;
        }
        // Allocate memory on the device
        int* dev_A, * dev_B, * dev_C;
        cudaMalloc(&dev_A, size);
        cudaMalloc(&dev_B, size);
        cudaMalloc(&dev_C, size);

        // Copy data from host to device
        cudaMemcpy(dev_A, A, size, cudaMemcpyHostToDevice);
        cudaMemcpy(dev_B, B, size, cudaMemcpyHostToDevice);

        // Launch the kernel
        int blockSize = 256;
        int numBlocks = (n + blockSize - 1) / blockSize;
        // Copy data from device to host
        cudaMemcpy(C, dev_C, size, cudaMemcpyDeviceToHost);

        // Print the results
        for (int i = 0; i < 10; i++)
        {
        cout << C[i] << " ";
```

```
        }
        cout << endl;

        // Free memory
        cudaFree(dev_A);
        cudaFree(dev_B);
        cudaFree(dev_C);
        cudaFreeHost(A);
        cudaFreeHost(B);
        cudaFreeHost(C);

        return 0;
}
```

Group A

Assignment 4(B)

Title of the Assignment: Write a Program for Matrix Multiplication using CUDA C

Objective of the Assignment: Students should be able to performProgram for Matrix Multiplication using CUDA C

Prerequisite:

1. CUDA Concept
2. Matrix Multiplication
3. How to execute Program in CUDA Environment

Contents for Theory:

1. **What is CUDA**
2. **Matrix Multiplication**
3. **Execution of CUDA Environment**

What is CUDA

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA. It allows developers to use the power of NVIDIA graphics processing units (GPUs) to accelerate computation tasks in various applications, including scientific computing, machine learning, and computer vision.CUDA provides a set of programming APIs, libraries, and tools that enable developers to write and execute parallel code on NVIDIA GPUs. It supports popular programming languages like C, C++, and Python, and provides a simple programming model that abstracts away much of the low-level details of GPU architecture.

Using CUDA, developers can exploit the massive parallelism and high computational power of GPUs to accelerate computationally intensive tasks, such as matrix operations, image processing, and deep learning. CUDA has become an important tool for scientific research and is widely used in fields like physics, chemistry, biology, and engineering.

Steps for Matrix Multiplication using CUDA

Here are the steps for implementing matrix multiplication using CUDA C:

1. Matrix Initialization: The first step is to initialize the matrices that you want to multiply. You can use standard C or CUDA functions to allocate memory for the matrices and initialize their values. The matrices are usually represented as 2D arrays.
2. Memory Allocation: The next step is to allocate memory on the host and the device for the matrices. You can use the standard C malloc function to allocate memory on the host and the CUDA function cudaMalloc() to allocate memory on the device.
3. Data Transfer: The third step is to transfer data between the host and the device. You can use the CUDA function cudaMemcpy() to transfer data from the host to the device or vice versa.
4. Kernel Launch: The fourth step is to launch the CUDA kernel that will perform the matrix multiplication on the device. You can use the <<<...>>> syntax to specify the number of blocks and threads to use. Each thread in the kernel will compute one element of the output matrix.
5. Device Synchronization: The fifth step is to synchronize the device to ensure that all kernel executions have completed before proceeding. You can use the CUDA function cudaDeviceSynchronize() to synchronize the device.
6. Data Retrieval: The sixth step is to retrieve the result of the computation from the device to the host. You can use the CUDA function cudaMemcpy() to transfer data from the device to the host.
7. Memory Deallocation: The final step is to deallocate the memory that was allocated on the host and the device. You can use the C free function to deallocate memory on the host and the CUDA function

cudaFree() to deallocate memory on the device.

Execution of Program over CUDA Environment

1. Install CUDA Toolkit: First, you need to install the CUDA Toolkit on your system. You can download the CUDA Toolkit from the NVIDIA website and follow the installation instructions provided.
2. Set up CUDA environment: Once the CUDA Toolkit is installed, you need to set up the CUDA environment on your system. This involves setting the PATH and LD_LIBRARY_PATH environment variables to the appropriate directories.
3. Write the CUDA program: You need to write a CUDA program that performs the addition of two large vectors. You can use a text editor to write the program and save it with a .cu extension.
4. Compile the CUDA program: You need to compile the CUDA program using the nvcc compiler that comes with the CUDA Toolkit. The command to compile the program is:

```
nvcc -o program_name program_name.cu
```

5. This will generate an executable program named program_name.

Run the CUDA program: Finally, you can run the CUDA program by executing the executable file generated in the previous step. The command to run the program is:

```
./program_name
```

Questions:

1. What are the advantages of using CUDA to perform matrix multiplication compared to using a CPU?
2. How do you handle matrices that are too large to fit in GPU memory in CUDA matrix multiplication?
3. How do you optimize the performance of the CUDA program for matrix multiplication?
4. How do you ensure correctness of the CUDA program for matrix multiplication and verify the results?

```
#include <cuda_runtime.h>
#include <iostream>
__global__ void matmul(int* A, int* B, int* C, int N) {
int Row = blockIdx.y*blockDim.y+threadIdx.y;
int Col = blockIdx.x*blockDim.x+threadIdx.x;
if (Row < N && Col < N) {
int Pvalue = 0;
for (int k = 0; k < N; k++) {
Pvalue += A[Row*N+k] * B[k*N+Col];
}
C[Row*N+Col] = Pvalue;
}
}
int main() {
int N = 512;
int size = N * N * sizeof(int);
int* A, * B, * C;
int* dev_A, * dev_B, * dev_C;
cudaMallocHost(&A, size);
cudaMallocHost(&B, size);
cudaMallocHost(&C, size);
cudaMalloc(&dev_A, size);
cudaMalloc(&dev_B, size);
cudaMalloc(&dev_C, size);
// Initialize matrices A and B
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
A[i*N+j] = i*N+j;
B[i*N+j] = j*N+i;
}
}
cudaMemcpy(dev_A, A, size,
cudaMemcpyHostToDevice);
cudaMemcpy(dev_B, B, size,
cudaMemcpyHostToDevice);
dim3 dimBlock(16, 16);
dim3 dimGrid(N/dimBlock.x, N/dimBlock.y);
matmul<<<dimGrid, dimBlock>>>(dev_A, dev_B,
dev_C, N);
cudaMemcpy(C, dev_C
// Print the result
for (int i = 0; i < 10; i++) {
for (int j = 0; j < 10; j++) {
std::cout << C[i*N+j] << " ";
}
std::cout << std::endl;
}
// Free memory
cudaFree(dev_A);
cudaFree(dev_B);
```

```
cudaFree(dev_C);
cudaFreeHost(A);
cudaFreeHost(B);
cudaFreeHost(C);
return 0;
```

Assignment No. 6

- **Title:** Linear regression by using Deep Neural network

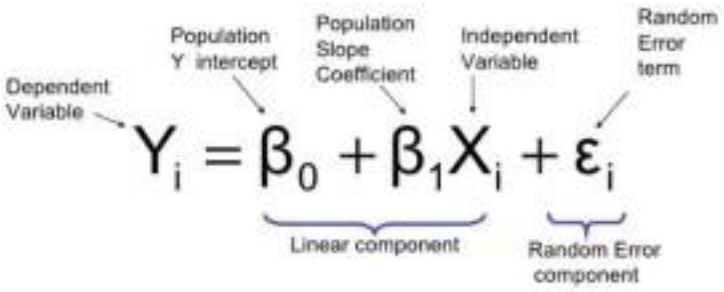
- **Objective:** Study and understand neural network by using Linear Regression for predicting house prices.

- **Problem Statement:** Implement Boston housing price prediction problem by Linear regression using Deep Neural network. Use Boston House price prediction dataset.

- **Theory:**

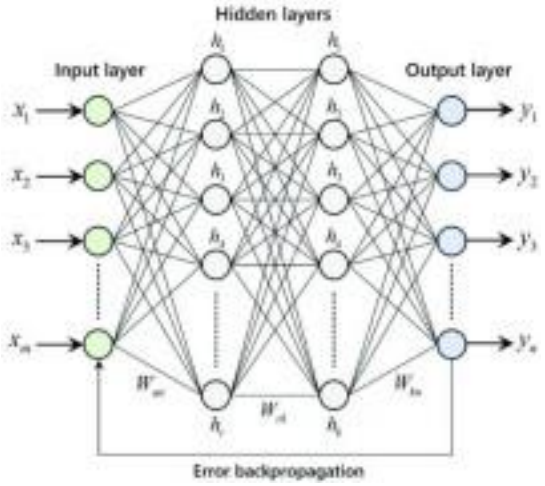
  Linear Regression

  Linear Regression is a supervised learning technique that involves learning the relationship between the features and the target. The target values are continuous, which means that the values can take any values between an interval. Use-cases of regression include stock market price prediction, house price prediction, sales prediction, and etc.



  Neural network

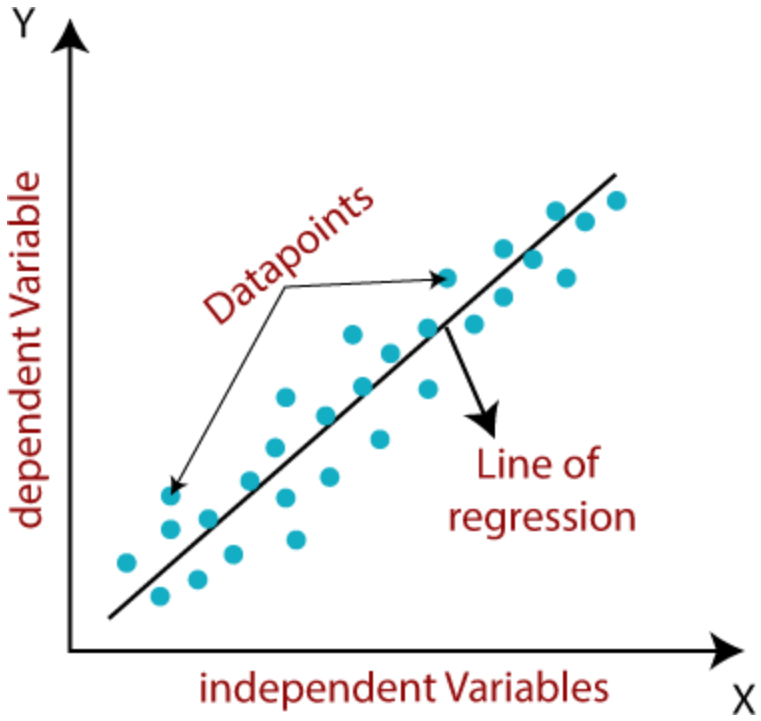
  Neural networks are formed when multiple neural layers combine with each other to give out a network, or we can say that there are some layers whose outputs are inputs for other layers.



The purpose of using Artificial Neural Networks for Regression over Linear Regression is that the linear regression can only learn the linear relationship between the features and target and therefore cannot learn the complex non-linear relationship. In order to learn the complex non-linear relationship between the features and target, we are in need of other techniques. One of those techniques is to use Artificial Neural Networks. Artificial Neural Networks have the ability to learn the complex relationship between the features and target due to the presence of activation function in each layer. Artificial Neural Networks are one of the deep learning algorithms that simulate the workings of neurons in the human brain.

Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (y) variables, hence called as linear regression. Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable.

The linear regression model provides a sloped straight line representing the relationship between the variables. Consider the below image:



Mathematically, we can represent a linear regression as:

$y= a_0+a_1x+ \varepsilon$

Here,

Y= Dependent Variable (Target Variable)
X= Independent Variable (predictor Variable)
a0= intercept of the line (Gives an additional degree of freedom)
a1 = Linear regression coefficient (scale factor to each input value).
ε = random error

The values for x and y variables are training datasets for Linear Regression model representation.

Types of Linear Regression

Linear regression can be further divided into two types of the algorithm:

- Simple Linear Regression:

If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.

- Multiple Linear regression:

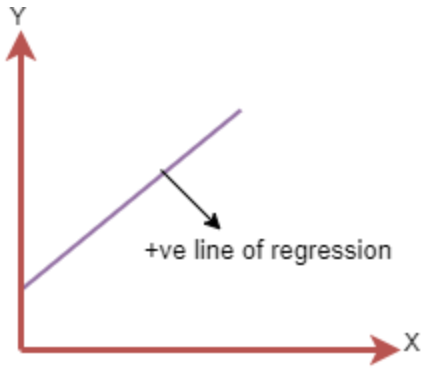
If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.

Linear Regression Line

A linear line showing the relationship between the dependent and independent variables is called a regression line. A regression line can show two types of relationship:

- Positive Linear Relationship:

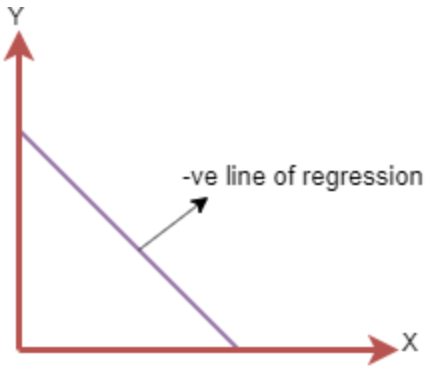
If the dependent variable increases on the Y-axis and independent variable increases on X-axis, then such a relationship is termed as a Positive linear relationship.



The line equation will be: $Y= a_0+a_1x$

- Negative Linear Relationship:

If the dependent variable decreases on the Y-axis and independent variable increases on the X-axis, then such a relationship is called a negative linear relationship.



The line of equation will be: $Y= -a_0+a_1x$

- **Dataset :** Boston Housing Dataset

- **References/Available link :**
  1.https://towardsdatascience.com/linear-regression-on-boston-housing-dataset-f409b7e4a155
  2.https://www.kaggle.com/code/shreayan98c/boston-house-price-prediction

- **Questions and answer :**
  Q1: What is Linear Regression?
  Q2: How would you detect Overfitting in Linear Models?
  Q3: What are types of Linear Regression?
  Q4: How can you check if the Regression model fits the data well?
  Q5: Define Linear Regression and its structure ?
  Q6: What is the difference between Mean Absolute Error (MAE) vs Mean Squared Error (MSE)?
  Q7: What's the difference between Covariance and Correlation?
  Q8: Explain the intuition behind Gradient Descent algorithm ?
  Q9: How is the Error calculated in a Linear Regression model?
  Q10: Why use Root Mean Squared Error (RMSE) instead of Mean Absolute Error (MAE)?

- **Mcqs:**
  1. The best fit line method for data in Linear Regression?
  A) Least Square Error
  B) Maximum Likelihood
  C) Logarithmic Loss
  D) Both A and B

2. Which of the following metrics can be used to evaluate a model with a continuous output variable?

A) Precision-Recall Curve
B) Accuracy
C) Logloss
D) Mean-Squared-Error

3. Which statement is true about outliers in Linear regression?

A) Linear regression model is not sensitive to outliers
B) Linear regression model is sensitive to outliers
C) Can't say
D) None of these

- **Conclusion :**

We have successfully solved Boston house price prediction problem by Linear Regression using deep neural networks.

In [1]:

```
# Importing the pandas for data processing and numpy for numerical computing
import numpy as np
import pandas as pd
```

In [2]:

```
# Importing the Boston Housing dataset from the sklearn
from sklearn.datasets import load_boston
boston = load_boston()
```

In [3]:

```
# Converting the data into pandas dataframe
data = pd.DataFrame(boston.data)
```

In [4]:

```
data.head()
```

Out[4]:

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33

In [5]:

```
# Adding the feature names to the dataframe
data.columns = boston.feature_names
```

In [6]:

```
# Adding the target variable to the dataset
data['PRICE'] = boston.target
```

In [7]:

```
# Looking at the data with names and target variable
data.head()
```

Out[7]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	L
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	

In [8]:

```
# Checking the null values in the dataset
data.isnull().sum()
```

Out[8]:

```
CRIM       0
ZN         0
INDUS      0
CHAS       0
NOX        0
RM         0
AGE        0
DIS        0
RAD        0
TAX        0
PTRATIO    0
B          0
LSTAT      0
PRICE      0
dtype: int64
```

In [9]:

```
# Checking the statistics of the data
data.describe()
```

Out[9]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.574901
std	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.148861
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900000
25%	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500	45.025000
50%	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500	77.500000
75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500	94.075000
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000000

In [10]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    float64
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    float64
 9   TAX      506 non-null    float64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  PRICE    506 non-null    float64
dtypes: float64(14)
memory usage: 55.5 KB
```

In [11]:

```
# X = data[['LSTAT','RM','PTRATIO']]
X = data.iloc[:,:-1]
y= data.PRICE
```

In [12]:

```
# Splitting the data into train and test for building the model
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.2, random_state =
```

In [13]:

```
# Linear Regression
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
```

In [14]:

```
# Fitting the model
regressor.fit(X_train,y_train)
```

Out[14]:

```
LinearRegression()
```

In [15]:

```
# Prediction on the test dataset
y_pred = regressor.predict(X_test)
```

In [16]:

```
# Predicting RMSE the Test set results
from sklearn.metrics import mean_squared_error
rmse = (np.sqrt(mean_squared_error(y_test, y_pred)))
print(rmse)
```

```
5.041784121402041
```

In [17]:

```
# Scaling the dataset
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [18]:

```
# Creating the neural network model
import keras
from keras.layers import Dense, Activation,Dropout
from keras.models import Sequential

model = Sequential()

model.add(Dense(128,activation  = 'relu',input_dim =13))
model.add(Dense(64,activation  = 'relu'))
model.add(Dense(32,activation  =  'relu'))
model.add(Dense(16,activation  =  'relu'))
model.add(Dense(1))
model.compile(optimizer = 'adam',loss = 'mean_squared_error')
```

In [19]:

```
model.fit(X_train, y_train, epochs = 100)
```

```
Epoch 24/100
13/13 [==============================]  - 0s 2ms/step - loss: 10.8369
Epoch 25/100
13/13 [==============================]  - 0s 3ms/step - loss: 10.7921
Epoch 26/100
13/13 [==============================]  - 0s 2ms/step - loss: 10.3413
Epoch 27/100
13/13 [==============================]  - 0s 3ms/step - loss: 10.2014
Epoch 28/100
13/13 [==============================]  - 0s 2ms/step - loss: 9.9532
Epoch 29/100
13/13 [==============================]  - 0s 2ms/step - loss: 9.8095
Epoch 30/100
13/13 [==============================]  - 0s 3ms/step - loss: 9.7494
Epoch 31/100
13/13 [==============================]  - 0s 4ms/step - loss: 9.6172
Epoch 32/100
13/13 [==============================]  - 0s 3ms/step - loss: 9.2688
Epoch 33/100
13/13 [==============================]  - 0s 3ms/step - loss: 9 2943
```

In [20]:

```
y_pred = model.predict(X_test)
```

```
4/4 [==============================] - 0s 4ms/step
```

In [27]:

```
# Predicting RMSE the Test set results
from sklearn.metrics import mean_squared_error
rmse = (np.sqrt(mean_squared_error(y_test, y_pred)))
print(rmse)
```

```
3.1410293739926494
```

Assignment No. 7

- **Title:** Classification using Deep neural network

- **Objective:** To study and understand how to solve classification problem using deep neural network.

- **Problem Statement:**Binary classification using Deep Neural Networks Example: Classify movie reviews into positive" reviews and "negative" reviews, just based on the text content of the reviews. Use IMDB dataset.

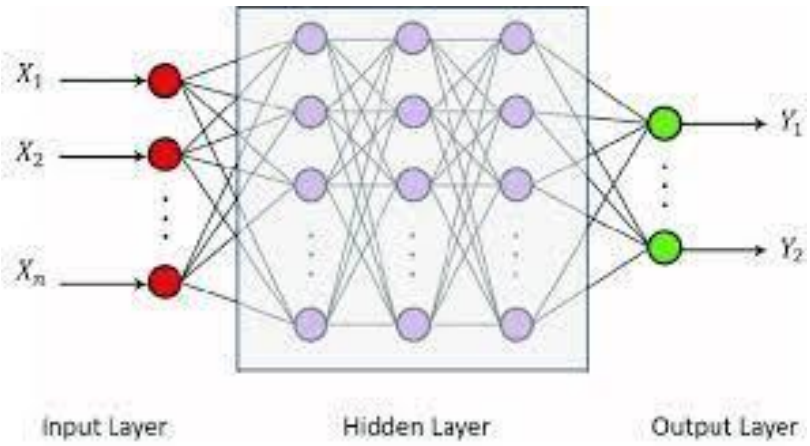
- **Theory:**

  Binary classification refers to those **classification tasks that have two class labels**. Examples include: Email spam detection (spam or not). Churn prediction (churn or not).

  Binary classification is one of the most common and frequently tackled problems in the machine learning domain. In it's simplest form **the user tries to classify an entity into one of the two possible categories**. For example, give the attributes of the fruits like weight, color, peel texture, etc.

  Deep learning (DL) is a subfield of machine learning based on learning multiple levels of representations by making a hierarchy of features where the higher levels are defined from the lower levels and the same lower level features can help in defining many higher level features. DL structure extends the traditional neural networks by adding more hidden layers to the network architecture between the input and output layers to model more complex and nonlinear relationships.

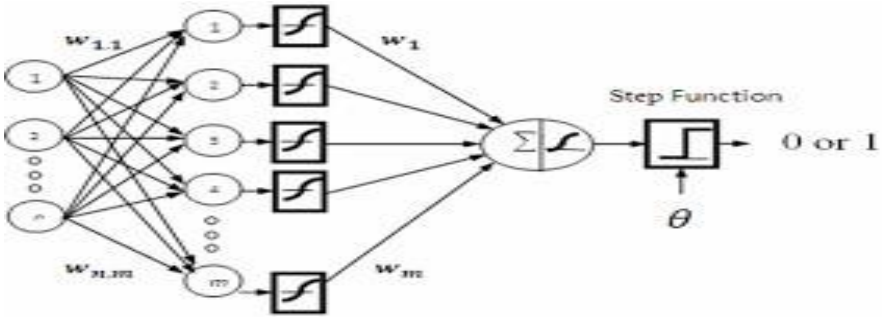
  Deep Neural Network is another DL architecture that is widely used for classification or regression with success in many areas. It's a typical feedforward network which the input flows from the input layer to the output layer through number of hidden layers which are more than two layers.



The main purpose of a neural network is to receive a set of inputs, perform progressively complex calculations on them, and give output to solve real world problems like classification.

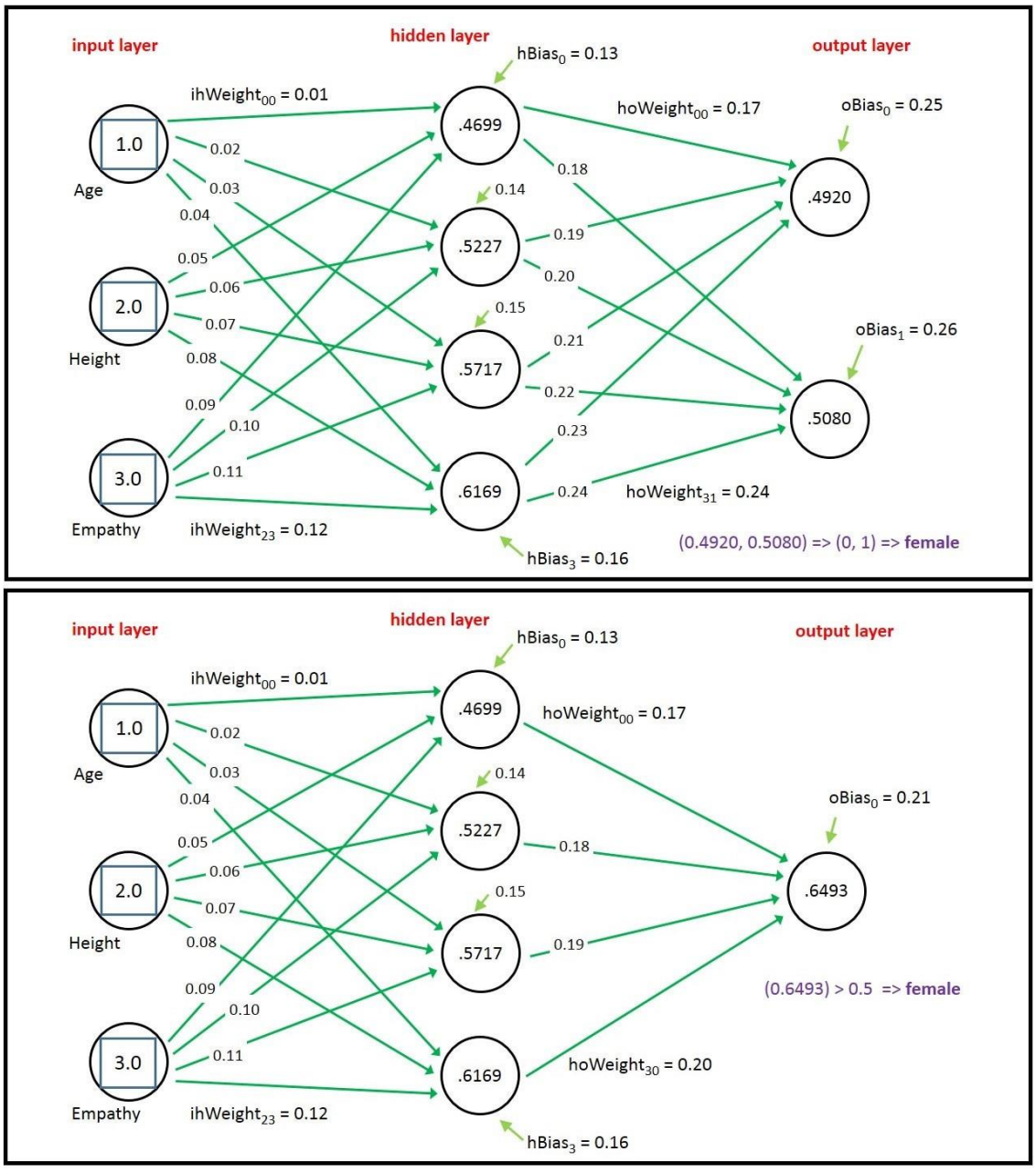
The IMDB sentiment classification dataset consists of 50,000 movie reviews from IMDB users that are labeled as either positive (1) or negative (0). The reviews are preprocessed and each one is encoded as a sequence of word indexes in the form of integers. The words within the reviews are indexed by their overall frequency within the dataset.

Building a neural network that performs binary classification involves making two simple changes:



- Add an activation function – specifically, the *sigmoid* activation function – to the output layer. Sigmoid reduces the output to a value from 0.0 to 1.0 representing a probability. For a reminder of what a sigmoid function does, see my post on binary classification.

- Change the loss function to *binary_crossentropy*, which is purpose-built for binary classifiers. Accordingly, change *metrics* to 'accuracy' so accuracies computed by the loss function are captured in the history object returned by *fit*.



- **Dataset :** IMDB dataset

- **References/Available link :**
  1. https://towardsdatascience.com/binary-classification-of-imdb-movie-reviews-648342bc70dd

2. https://vsokolov.org/courses/41000/notes/nlp-imdb.html

- **Questions and answer :**
  1. What is Deep Learning?
  2. What is a Neural network?
  3. What is Binary Classification?
  4. What types of Classification Algorithms do you know?
  5. Can you choose a classifier based on the size of the training set?
  6. Could you convert Regression into Classification and vice versa?
  7. How do you use a supervised Logistic Regression for Classification?
  8. How does ROC curve and AUC value help measure how good a model is?
  9. Name some classification metrics and when would you use each one?
  10. What's the difference between Generative Classifiers and Discriminative Classifiers? Name some examples of each one?

- **Mcqs:**
  1. Following are the types of supervised learning_____
     a.regression
     b.classification
     c.subgroup discovery
     d.All of above

  2. Following are the descriptive models_____
     a.classification
     b.clustering
     c.association rule
     d.Both 1 and 2

  3. In binary classification number of classes must be_____
     a.equals to two
     b.less than two
     c.greater than two
     d.None

- **Conclusion :**We have successfully classified movie reviews using deep neural networks.

In [34]:

```
from keras.datasets import imdb

# Load the data, keeping only 10,000 of the most frequently occuring words
(train_data, train_labels), (test_data, test_labels) = imdb.load_data(num_words = 10000)
```

In [35]:

```
# Here is a list of maximum indexes in every review
print(type([max(sequence) for sequence in train_data]))

# Find the maximum of all max indexes
max([max(sequence) for sequence in train_data])
```

```
<class 'list'>
```

Out[35]:

```
9999
```

In [36]:

```
# step 1: load the dictionary mappings from word to integer index
word_index = imdb.get_word_index()

# step 2: reverse word index to map integer indexes to their respective words
reverse_word_index = dict([(value, key) for (key, value) in word_index.items()])

# Step 3: decode the review, mapping integer indices to words
#
# indices are off by 3 because 0, 1, and 2 are reserverd indices for "padding", "Start of
decoded_review = ' '.join([reverse_word_index.get(i-3, '?') for i in train_data[0]])

decoded_review
```

Out[36]:

```
"? this film was just brilliant casting location scenery story direction e
veryone's really suited the part they played and you could just imagine be
ing there robert ? is an amazing actor and now the same being director ? f
ather came from the same scottish island as myself so i loved the fact the
re was a real connection with this film the witty remarks throughout the f
ilm were great it was just brilliant so much that i bought the film as soo
n as it was released for ? and would recommend it to everyone to watch and
the fly fishing was amazing really cried at the end it was so sad and you
know what they say if you cry at a film it must have been good and this de
finitely was also ? to the two little boy's that played the ? of norman an
d paul they were just brilliant children are often left out of the ? list
i think because the stars that play them all grown up are such a big profi
le for the whole film but these children are amazing and should be praised
for what they have done don't you think the whole story was so lovely beca
use it was true and was someone's life after all that was shared with us a
ll"
```

In [37]:

```
# Vectorize input data

import numpy as np

def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))    # Creates an all zero matrix of sh
    for i,sequence in enumerate(sequences):
        results[i,sequence] = 1                        # Sets specific indices of results
    return results

# Vectorize training Data
X_train = vectorize_sequences(train_data)

# Vectorize testing Data
X_test = vectorize_sequences(test_data)
```

In [38]:

```
X_train[0]
```

Out[38]:

```
array([0., 1., 1., ..., 0., 0., 0.])
```

In [39]:

```
X_train.shape
```

Out[39]:

```
(25000, 10000)
```

In [40]:

```
# Vectorize labels
y_train = np.asarray(train_labels).astype('float32')
y_test  = np.asarray(test_labels).astype('float32')
```

In [41]:

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

In [42]:

```
from keras import optimizers
from keras import losses
from keras import metrics

model.compile(optimizer=optimizers.RMSprop(lr=0.001), loss = losses.binary_crossentropy,
```

```
/usr/local/lib/python3.8/dist-packages/keras/optimizers/optimizer_v2/rmspr
op.py:135: UserWarning: The `lr` argument is deprecated, use `learning_rat
e` instead.
  super(RMSprop, self).__init__(name, **kwargs)
```

In [43]:

```
# Input for Validation
X_val = X_train[:10000]
partial_X_train = X_train[10000:]

# Labels for validation
y_val = y_train[:10000]
partial_y_train = y_train[10000:]
```

In [44]:

```
history = model.fit(partial_X_train, partial_y_train, epochs=20, batch_size=512, validati
```

```
Epoch 1/20
30/30 [==============================] - 2s 56ms/step - loss: 0.4878 - bin
ary_accuracy: 0.7969 - val_loss: 0.3705 - val_binary_accuracy: 0.8656
Epoch 2/20
30/30 [==============================] - 1s 45ms/step - loss: 0.2905 - bin
ary_accuracy: 0.9047 - val_loss: 0.3232 - val_binary_accuracy: 0.8687
Epoch 3/20
30/30 [==============================] - 2s 54ms/step - loss: 0.2151 - bin
ary_accuracy: 0.9300 - val_loss: 0.2884 - val_binary_accuracy: 0.8847
Epoch 4/20
30/30 [==============================] - 1s 42ms/step - loss: 0.1678 - bin
ary_accuracy: 0.9447 - val_loss: 0.2873 - val_binary_accuracy: 0.8839
Epoch 5/20
30/30 [==============================] - 2s 61ms/step - loss: 0.1391 - bin
ary_accuracy: 0.9546 - val_loss: 0.2910 - val_binary_accuracy: 0.8835
Epoch 6/20
30/30 [==============================] - 2s 63ms/step - loss: 0.1157 - bin
ary_accuracy: 0.9634 - val_loss: 0.2987 - val_binary_accuracy: 0.8840
Epoch 7/20
30/30 [==============================] - 1s 41ms/step - loss: 0.0975 - bin
ary_accuracy: 0.9701 - val_loss: 0.3287 - val_binary_accuracy: 0.8774
Epoch 8/20
30/30 [==============================] - 1s 34ms/step - loss: 0.0750 - bin
ary_accuracy: 0.9795 - val_loss: 0.3497 - val_binary_accuracy: 0.8790
Epoch 9/20
30/30 [==============================] - 1s 35ms/step - loss: 0.0700 - bin
ary_accuracy: 0.9807 - val_loss: 0.3777 - val_binary_accuracy: 0.8768
Epoch 10/20
30/30 [==============================] - 1s 34ms/step - loss: 0.0592 - bin
ary_accuracy: 0.9826 - val_loss: 0.3836 - val_binary_accuracy: 0.8769
Epoch 11/20
30/30 [==============================] - 1s 35ms/step - loss: 0.0467 - bin
ary_accuracy: 0.9881 - val_loss: 0.4225 - val_binary_accuracy: 0.8697
Epoch 12/20
30/30 [==============================] - 1s 35ms/step - loss: 0.0396 - bin
ary_accuracy: 0.9909 - val_loss: 0.4334 - val_binary_accuracy: 0.8740
Epoch 13/20
30/30 [==============================] - 1s 33ms/step - loss: 0.0306 - bin
ary_accuracy: 0.9935 - val_loss: 0.4850 - val_binary_accuracy: 0.8631
Epoch 14/20
30/30 [==============================] - 1s 33ms/step - loss: 0.0266 - bin
ary_accuracy: 0.9948 - val_loss: 0.4939 - val_binary_accuracy: 0.8720
Epoch 15/20
30/30 [==============================] - 1s 35ms/step - loss: 0.0208 - bin
ary_accuracy: 0.9965 - val_loss: 0.5268 - val_binary_accuracy: 0.8700
Epoch 16/20
30/30 [==============================] - 1s 34ms/step - loss: 0.0172 - bin
ary_accuracy: 0.9977 - val_loss: 0.5638 - val_binary_accuracy: 0.8683
Epoch 17/20
30/30 [==============================] - 1s 48ms/step - loss: 0.0141 - bin
ary_accuracy: 0.9980 - val_loss: 0.5940 - val_binary_accuracy: 0.8655
Epoch 18/20
30/30 [==============================] - 2s 52ms/step - loss: 0.0117 - bin
ary_accuracy: 0.9983 - val_loss: 0.6302 - val_binary_accuracy: 0.8639
Epoch 19/20
30/30 [==============================] - 1s 40ms/step - loss: 0.0102 - bin
ary_accuracy: 0.9985 - val_loss: 0.6597 - val_binary_accuracy: 0.8657
Epoch 20/20
30/30 [==============================] - 1s 34ms/step - loss: 0.0080 - bin
ary_accuracy: 0.9989 - val_loss: 0.6970 - val_binary_accuracy: 0.8651
```

In [45]:

```
history_dict = history.history
history_dict.keys()
```

Out[45]:

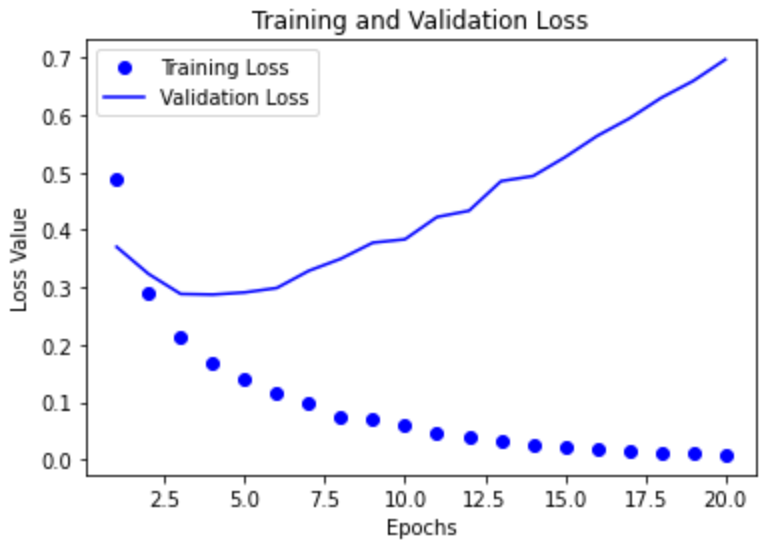
```
dict_keys(['loss', 'binary_accuracy', 'val_loss', 'val_binary_accuracy'])
```

In [46]:

```
import matplotlib.pyplot as plt
%matplotlib inline
```

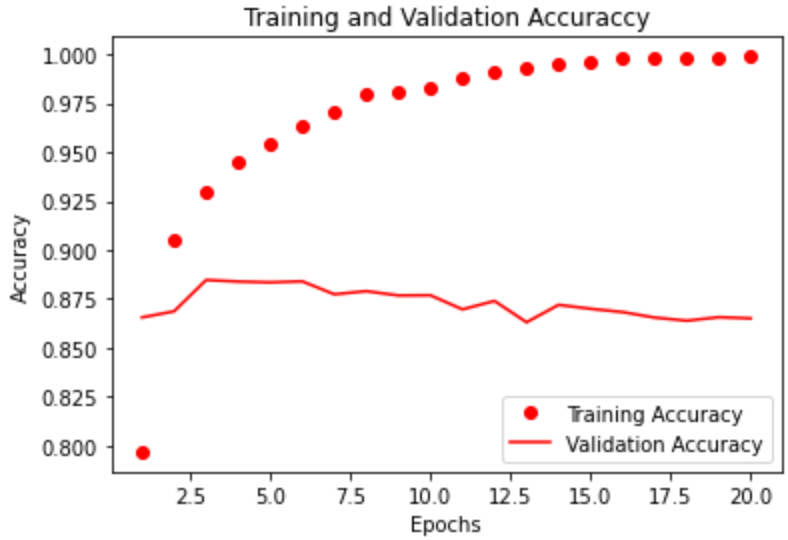
In [47]:

```
# Plotting losses
loss_values = history_dict['loss']
val_loss_values = history_dict['val_loss']

epochs = range(1, len(loss_values) + 1)

plt.plot(epochs, loss_values, 'bo', label="Training Loss")
plt.plot(epochs, val_loss_values, 'b', label="Validation Loss")

plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss Value')
plt.legend()

plt.show()
```



In [48]:

```
# Training and Validation Accuracy

acc_values = history_dict['binary_accuracy']
val_acc_values = history_dict['val_binary_accuracy']

epochs = range(1, len(loss_values) + 1)

plt.plot(epochs, acc_values, 'ro', label="Training Accuracy")
plt.plot(epochs, val_acc_values, 'r', label="Validation Accuracy")

plt.title('Training and Validation Accuraccy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```



In [49]:

```
model.fit(partial_X_train, partial_y_train, epochs=3, batch_size=512, validation_data=(X_
```

```
Epoch 1/3
30/30 [==============================] - 2s 61ms/step - loss: 0.0040 - bin
ary_accuracy: 0.9999 - val_loss: 0.7310 - val_binary_accuracy: 0.8648
Epoch 2/3
30/30 [==============================] - 1s 33ms/step - loss: 0.0061 - bin
ary_accuracy: 0.9988 - val_loss: 0.7557 - val_binary_accuracy: 0.8636
Epoch 3/3
30/30 [==============================] - 1s 34ms/step - loss: 0.0039 - bin
ary_accuracy: 0.9995 - val_loss: 0.8755 - val_binary_accuracy: 0.8475
```

Out[49]:

```
<keras.callbacks.History at 0x7f1cc77b7880>
```

In [50]:

```
# Making Predictions for testing data
np.set_printoptions(suppress=True)
result = model.predict(X_test)
```

```
782/782 [==============================] - 2s 2ms/step
```

In [51]:

```
result
```

Out[51]:

```
array([[0.00190504],
       [1.        ],
       [0.09390965],
       ...,
       [0.00027408],
       [0.00282606],
       [0.3104798 ]], dtype=float32)
```

In [52]:

```
y_pred = np.zeros(len(result))
for i, score in enumerate(result):
    y_pred[i] = 1 if score > 0.5 else 0
```

In [53]:

```
from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(y_pred, y_test)
```

In [54]:

```
# error
mae
```

Out[54]:

```
0.16668
```

Assignment No. 8

- **Title:** Convolutional neural network (CNN)

- **Objective:** Study and understand Convolutional Neural Network by creating a classifier.

- **Problem Statement:**Use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories.

- **Theory:**

  CNNs are a class of Deep Neural Networks that can recognize and classify particular features from images and are widely used for analyzing visual images. Their applications range from image and video recognition, image classification, medical image analysis, computer vision and natural language processing.

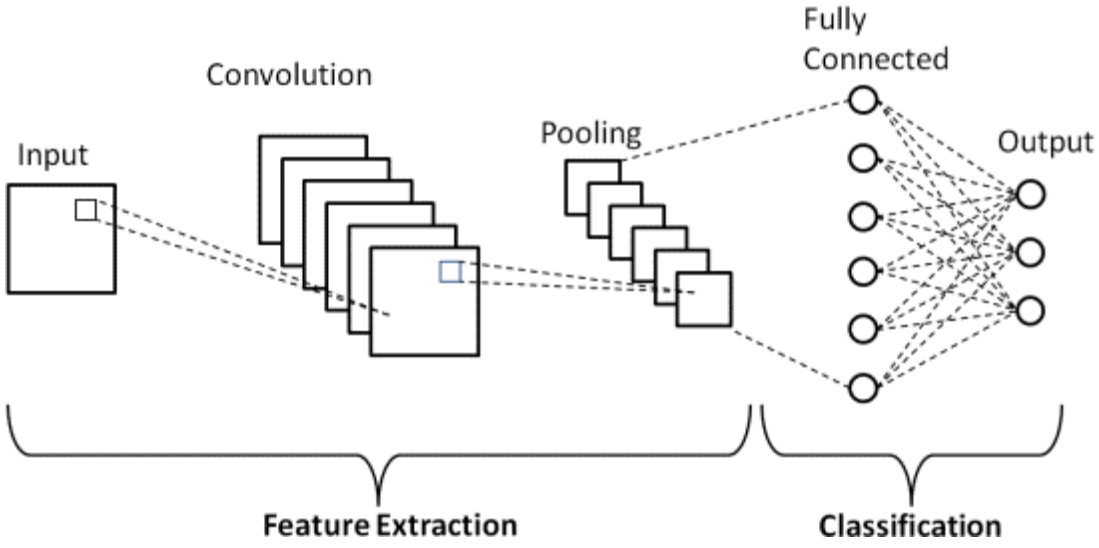
  CNN has high accuracy, and because of the same, it is useful in image recognition. Image recognition has a wide range of uses in various industries such as medical image analysis, phone, security, recommendation systems, etc.

  The term 'Convolution" in CNN denotes the mathematical function of convolution which is a special kind of linear operation wherein two functions are multiplied to produce a third function which expresses how the shape of one function is modified by the other. In simple terms, two images which can be represented as matrices are multiplied to give an output that is used to extract features from the image.

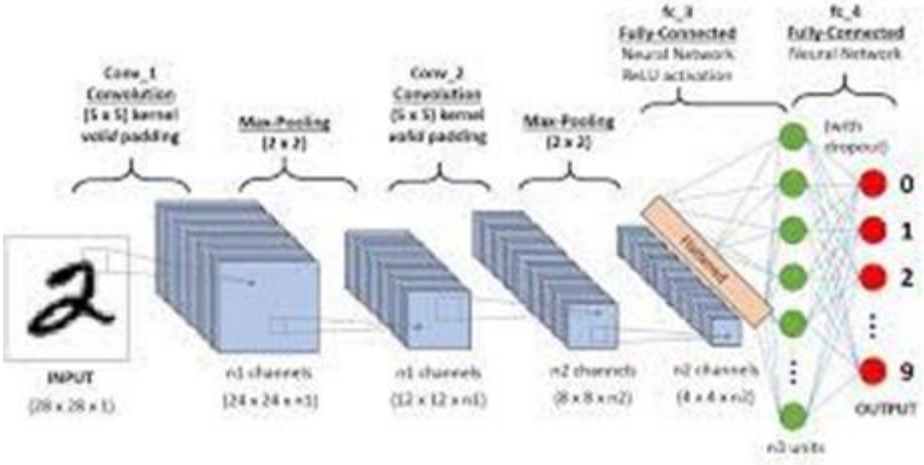
  ## Basic Architecture

  There are two main parts to a CNN architecture

  - A convolution tool that separates and identifies the various features of the image for analysis in a process called Feature Extraction.
  - The network of feature extraction consists of many pairs of convolutional or pooling layers.
  - A fully connected layer that utilizes the output from the convolution process and predicts the class of the image based on the features extracted in previous stages.
  - This CNN model of feature extraction aims to reduce the number of features present in a dataset. It creates new features which summarizes the existing features contained in an original set of features. There are many CNN layers as shown in the CNN architecture diagram.



Convolutional neural networks are distinguished from other neural networks by their superior performance with image, speech, or audio signal inputs. They have three main types of layers, which are:
Convolutional layer
Pooling layer
Fully-connected (FC) layer



1. Convolutional Layer

This layer is the first layer that is used to extract the various features from the input images. In this layer, the mathematical operation of convolution is performed between the input image and a filter of a particular size MxM. By sliding the filter over the input image, the dot product is taken between the filter and the parts of the input image with respect to the size of the filter (MxM).

The output is termed as the Feature map which gives us information about the image such as the corners and edges. Later, this feature map is fed to other layers to learn several other features of the input image.
The convolution layer in CNN passes the result to the next layer once applying the convolution operation in the input. Convolutional layers in CNN benefit a lot as they ensure the spatial relationship between the pixels is intact.

2. Pooling Layer

In most cases, a Convolutional Layer is followed by a Pooling Layer. The primary aim of this layer is to decrease the size of the convolved feature map to reduce the computational costs. This is performed by decreasing the connections between layers and independently operates on each feature map. Depending upon method used, there are several types of Pooling operations. It basically summarises the features generated by a convolution layer.
In Max Pooling, the largest element is taken from feature map. Average Pooling calculates the average of the elements in a predefined sized Image section. The total sum of the elements in the predefined section is computed in Sum Pooling. The Pooling Layer usually serves as a bridge between the Convolutional Layer and the FC Layer.
This CNN model generalises the features extracted by the convolution layer, and helps the networks to recognise the features independently. With the help of this, the computations are also reduced in a network.

3. Fully Connected Layer

The Fully Connected (FC) layer consists of the weights and biases along with the neurons and is used to connect the neurons between two different layers. These layers are usually placed before the output layer and form the last few layers of a CNN Architecture.
In this, the input image from the previous layers are flattened and fed to the FC layer. The flattened vector then undergoes few more FC layers where the mathematical functions operations usually take place. In this stage, the classification process begins to take place. The reason two layers are connected is that two fully connected layers will perform better than a single connected layer. These layers in CNN reduce the human supervision

There are two main types of pooling:
Max pooling: As the filter moves across the input, it selects the pixel with the maximum value to send to the output array. As an aside, this approach tends to be used more often compared to average pooling.

Average pooling: As the filter moves across the input, it calculates the average value within the receptive field to send to the output array.
The name of the full-connected layer aptly describes itself. The pixel values of the input image are not directly connected to the output layer in partially connected layers.
However, in the fully-connected layer, each node in the output layer connects directly to a node in the previous layer. This layer performs the task of classification based on the features extracted through the previous layers and their different filters. While

convolutional and pooling layers tend to use ReLu functions, FC layers usually leverage a softmax activation function to classify inputs appropriately, producing a probability from 0 to 1.

- **Dataset :** MNIST Fashion Dataset.

- **References/Available link :**
  1. https://machinelearningmastery.com/how-to-develop-a-cnn-from-scratch-for-fashion-mnist-clothing-classification/
  2. https://www.kaggle.com/code/anindya2906/fashion-mnist-classification

- **Questions and answer :**
  1. What do you mean by Convolutional Neural Network?
  2. Why do we prefer Convolutional Neural networks (CNN) over Artificial Neural networks (ANN) for image data as input?
  3. Explain the different layers in CNN?
  4. Explain the significance of the RELU Activation function in Convolution Neural Network?
  5. Why do we use a Pooling Layer in a CNN?
  6. What is the size of the feature map for a given input size image, Filter Size, Stride, and Padding amount?
  7. Explain the terms "Valid Padding" and "Same Padding" in CNN?
  8. What are the different types of Pooling? Explain their characteristics?
  9. Does the size of the feature map always reduce upon applying the filters? Explain why or why not?
  10. What is Stride? What is the effect of high Stride on the feature map?

- **Mcqs:**
  1. Which is the most direct application of neural networks?
  a) vector quantization
  b) pattern mapping
  c) pattern classification
  d) control applications

  2. What are pros of neural networks over computers?
  a) they have ability to learn b examples
  b) they have real time high computational rates
  c) they have more tolerance
  d) all of the mentioned

3. which of the following is false?
a) neural networks are artificial copy of the human brain
b) neural networks have high computational rates than conventional computers
c) neural networks learn by examples
d) none of the mentioned

- **Conclusion :**
  We have successfully created a classifier using Convolutional neural network.

In [25]:

```
from __future__ import absolute_import, division, print_function

# TensorFlow and tf.keras
import tensorflow as tf
from tensorflow import keras

# Helper libraries
import numpy as np
import matplotlib.pyplot as plt
```

In [26]:

```
fashion_mnist = keras.datasets.fashion_mnist

(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
```

In [27]:

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

In [28]:

```
train_images.shape
```

Out[28]:

```
(60000, 28, 28)
```

In [29]:

```
len(train_labels)
```

Out[29]:

```
60000
```

In [30]:

```
train_labels
```

Out[30]:

```
array([9, 0, 0, ..., 3, 0, 5], dtype=uint8)
```

In [31]:

```
test_images.shape
```

Out[31]:

```
(10000, 28, 28)
```

In [32]:

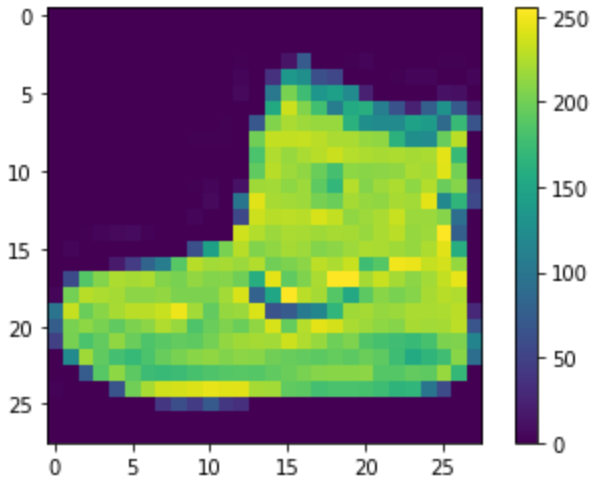
```
len(test_labels)
```

Out[32]:

```
10000
```

In [33]:

```
plt.figure()
plt.imshow(train_images[0])
plt.colorbar()
plt.grid(False)
plt.show()
```



In [34]:

```
train_images = train_images / 255.0

test_images = test_images / 255.0
```

In [35]:

```
plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[train_labels[i]])
plt.show()
```

In [36]:

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation=tf.nn.relu),
    keras.layers.Dense(10, activation=tf.nn.softmax)
])
```

In [37]:

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

In [38]:

```
model.fit(train_images, train_labels, epochs=5)
```

```
Epoch 1/5
1875/1875 [==============================] - 7s 4ms/step - loss: 0.4971 -
accuracy: 0.8257
Epoch 2/5
1875/1875 [==============================] - 6s 3ms/step - loss: 0.3751 -
accuracy: 0.8648
Epoch 3/5
1875/1875 [==============================] - 7s 4ms/step - loss: 0.3338 -
accuracy: 0.8780
Epoch 4/5
1875/1875 [==============================] - 7s 3ms/step - loss: 0.3104 -
accuracy: 0.8856
Epoch 5/5
1875/1875 [==============================] - 8s 4ms/step - loss: 0.2946 -
accuracy: 0.8906
```

Out[38]:

```
<keras.callbacks.History at 0x7f07800931f0>
```

In [39]:

```
test_loss, test_acc = model.evaluate(test_images, test_labels)

print('Test accuracy:', test_acc)
```

```
313/313 [==============================] - 1s 2ms/step - loss: 0.3542 - ac
curacy: 0.8736
Test accuracy: 0.8736000061035156
```

In [40]:

```
predictions = model.predict(test_images)
```

```
313/313 [==============================] - 1s 2ms/step
```

In [41]:

```
predictions[0]
```

Out[41]:

```
array([4.9921914e-06, 3.3841974e-07, 1.8965457e-07, 1.1019566e-09,
       1.3401749e-06, 4.8021390e-03, 1.4439345e-06, 5.2010346e-02,
       2.6271462e-05, 9.4315302e-01], dtype=float32)
```

In [42]:

```
np.argmax(predictions[0])
```

Out[42]:

```
9
```

In [43]:

```
test_labels[0]
```

Out[43]:

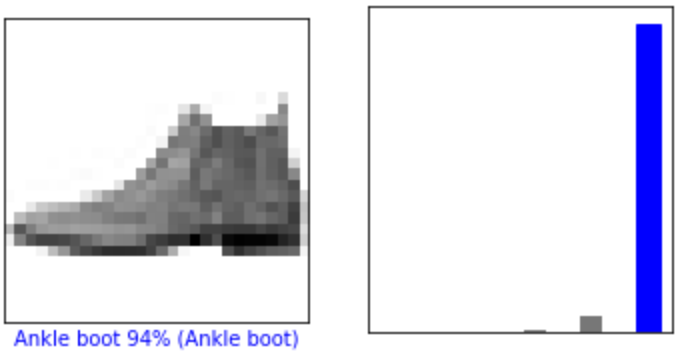
```
9
```

In [44]:

```
def plot_image(i, predictions_array, true_label, img):
  predictions_array, true_label, img = predictions_array[i], true_label[i], img[i]
  plt.grid(False)
  plt.xticks([])
  plt.yticks([])

  plt.imshow(img, cmap=plt.cm.binary)

  predicted_label = np.argmax(predictions_array)
  if predicted_label == true_label:
    color = 'blue'
  else:
    color = 'red'

  plt.xlabel("{} {:2.0f}% ({})".format(class_names[predicted_label],
                                100*np.max(predictions_array),
                                class_names[true_label]),
                                color=color)

def plot_value_array(i, predictions_array, true_label):
  predictions_array, true_label = predictions_array[i], true_label[i]
  plt.grid(False)
  plt.xticks([])
  plt.yticks([])
  thisplot = plt.bar(range(10), predictions_array, color="#777777")
  plt.ylim([0, 1])
  predicted_label = np.argmax(predictions_array)

  thisplot[predicted_label].set_color('red')
  thisplot[true_label].set_color('blue')
```

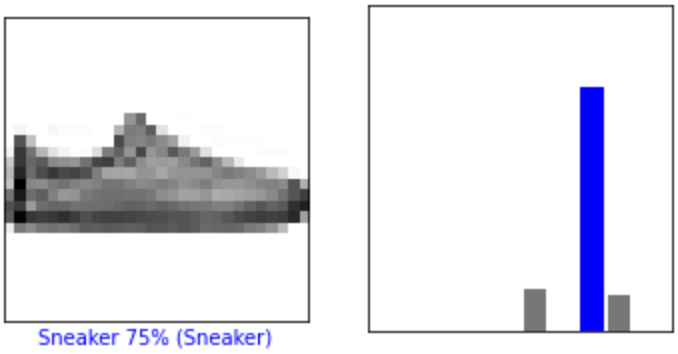
In [45]:

```
i = 0
plt.figure(figsize=(6,3))
plt.subplot(1,2,1)
plot_image(i, predictions, test_labels, test_images)
plt.subplot(1,2,2)
plot_value_array(i, predictions, test_labels)
plt.show()
```



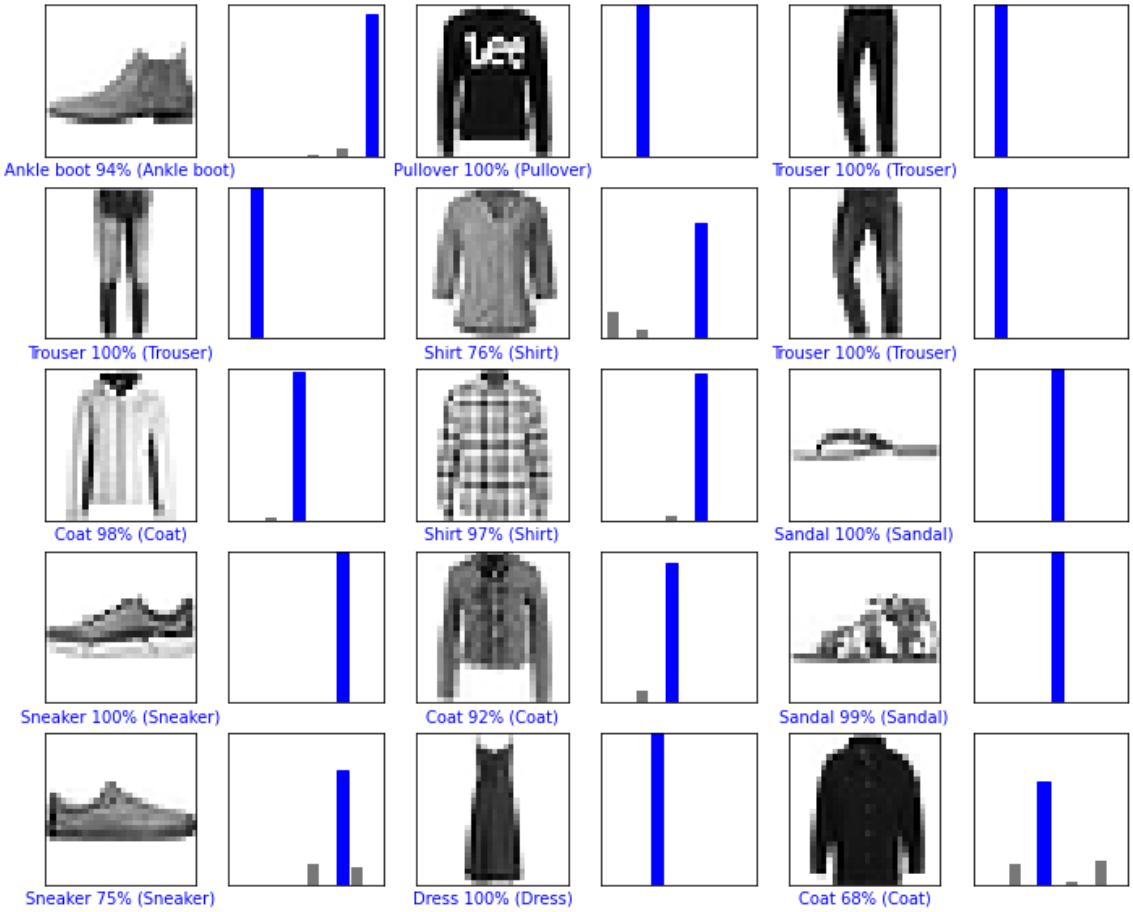
In [46]:

```
i = 12
plt.figure(figsize=(6,3))
plt.subplot(1,2,1)
plot_image(i, predictions, test_labels, test_images)
plt.subplot(1,2,2)
plot_value_array(i, predictions, test_labels)
plt.show()
```



In [47]:

```
# Plot the first X test images, their predicted label, and the true label
# Color correct predictions in blue, incorrect predictions in red
num_rows = 5
num_cols = 3
num_images = num_rows*num_cols
plt.figure(figsize=(2*2*num_cols, 2*num_rows))
for i in range(num_images):
  plt.subplot(num_rows, 2*num_cols, 2*i+1)
  plot_image(i, predictions, test_labels, test_images)
  plt.subplot(num_rows, 2*num_cols, 2*i+2)
  plot_value_array(i, predictions, test_labels)
plt.show()
```



In [48]:

```
# Grab an image from the test dataset
img = test_images[0]

print(img.shape)
```

```
(28, 28)
```

In [49]:

```
# Add the image to a batch where it's the only member.
img = (np.expand_dims(img,0))

print(img.shape)
```

```
(1, 28, 28)
```

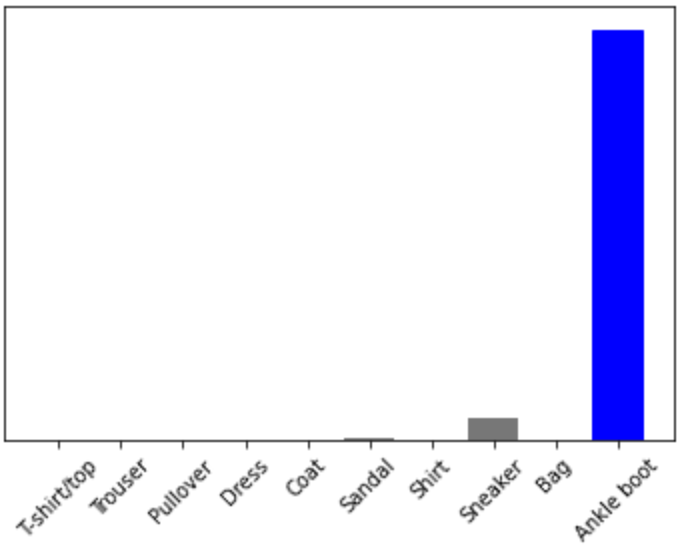
In [50]:

```
predictions_single = model.predict(img)

print(predictions_single)
```

```
1/1 [==============================] - 0s 25ms/step
[[4.9921905e-06 3.3841968e-07 1.8965453e-07 1.1019544e-09 1.3401745e-06
  4.8021362e-03 1.4439315e-06 5.2010350e-02 2.6271480e-05 9.4315284e-01]]
```

In [51]:

```
plot_value_array(0, predictions_single, test_labels)
_ = plt.xticks(range(10), class_names, rotation=45)
```



In [52]:

```
np.argmax(predictions_single[0])
```

Out[52]:

```
9
```

Assignment No. 9

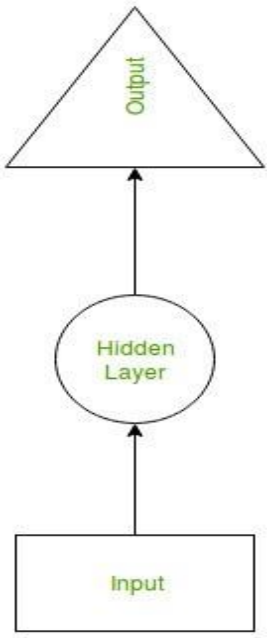
- **Title:** Recurrent neural network (RNN) .

- **Objective:** To study and understand Recurrent Neural Network by doing analysis and designing prediction systems.

- **Problem Statement:**Use the Google stock prices dataset and design a time series analysis and prediction system using RNN.

- **Theory:**

Recurrent Neural Network(RNN) is a type of Neural Network where the output from the previous step are fed as input to the current step. In traditional neural networks, all the inputs and outputs are independent of each other, but in cases like when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words. Thus RNN came into existence, which solved this issue with the help of a Hidden Layer. The main and most important feature of RNN is Hidden state, which remembers some information about a sequence.



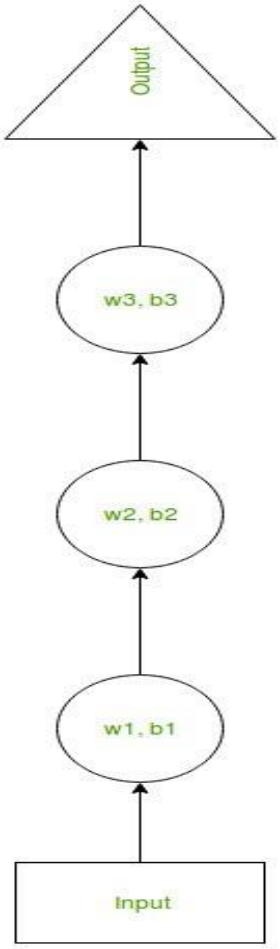
RNN have a "memory" which remembers all information about what has been calculated. It uses the same parameters for each input as it performs the same task on all the inputs or hidden

layers to produce the output. This reduces the complexity of parameters, unlike other neural networks.

How RNN works

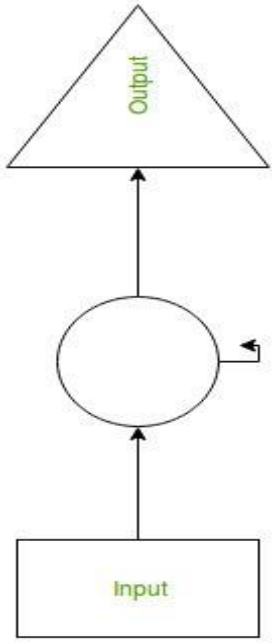
The working of an RNN can be understood with the help of the below example:

Example: Suppose there is a deeper network with one input layer, three hidden layers, and one output layer. Then like other neural networks, each hidden layer will have its own set of weights and biases, let's say, for hidden layer 1 the weights and biases are (w1, b1), (w2, b2) for the second hidden layer, and (w3, b3) for the third hidden layer. This means that each of these layers is independent of the other, i.e. they do not memorize the previous outputs.



Now the RNN will do the following:

- RNN converts the independent activations into dependent activations by providing the same weights and biases to all the layers, thus reducing the complexity of increasing parameters and memorizing each previous output by giving each output as input to the next hidden layer.
- Hence these three layers can be joined together such that the weights and bias of all the hidden layers are the same, in a single recurrent layer.



$$h_t = f(h_{t-1}, x_t)$$

The formula for calculating cur rent state: where:
ht -> current state
ht-1 -> previous state
xt -> input state
Formula for applying Activation function(tanh): wh

$$h_t = \tanh (W_{hh}h_{t-1} + W_{xh}x_t)$$

ere:

whh -> weight at recurrent neuron
wxh -> weight at input neuron
The formula for calculating output:

$$y_t = W_{hy}h_t$$

Yt -> output
Why -> weight at output layer
Training through RNN

1. A single-time step of the input is provided to the network.
2. Then calculate its current state using a set of current input and the previous state.
3. The current ht becomes ht-1 for the next time step.
4. One can go as many time steps according to the problem and join the information from all the previous states.

5. Once all the time steps are completed the final current state is used to calculate the output.
6. The output is then compared to the actual output i.e the target output and the error is generated.
7. The error is then back-propagated to the network to update the weights and hence the network (RNN) is trained.

Advantages of Recurrent Neural Network

1. An RNN remembers each and every piece of information through time. It is useful in time series prediction only because of the feature to remember previous inputs as well. This is called Long Short Term Memory.
2. Recurrent neural networks are even used with convolutional layers to extend the effective pixel neighborhood.

Disadvantages of Recurrent Neural Network

1. Gradient vanishing and exploding problems.
2. Training an RNN is a very difficult task.
3. It cannot process very long sequences if using tanh or relu as an activation function.

Applications of Recurrent Neural Network

1. Language Modelling and Generating Text
2. Speech Recognition
3. Machine Translation
4. Image Recognition, Face detection
5. Time series Forecasting

Long short-term memory (LSTM)
This is a popular RNN architecture as a solution to vanishing gradient problem. It addresses the problem of long-term dependencies. That is, if the previous state that is influencing the current prediction is not in the recent past, the RNN model may not be able to accurately predict the current state. LSTMs have "cells" in the hidden layers of the neural network, which have three gates–an input gate, an output gate, and a forget gate. These gates control the flow of information which is needed to predict the output in the network.

- **Dataset :** Google stock prices dataset

- **References/Available link**
  1. https://towardsdatascience.com/lstm-for-google-stock-price-prediction-e35f5cc84165
  2. https://medium.com/linkit-intecs/developing-a-recurrent-neural-network-rnn-for-a-google-stock-price-dataset-having-continuous-data-97b6e4e724a3

- **Questions and answer :**

1. What's the difference between Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) and in which cases would use each one?
2. How many dimensions must the inputs of an RNN layer have?
3. What are the main difficulties when training RNNs? How can you handle them?
4. What are the uses of using RNN in NLP?
5. What types of Recurrent Neural Networks (RNN) do you know?
6. What's the difference between Stateful RNN vs Stateless RNN? What are their pros and cons?
7. What's the difference between Traditional Feedforward Networks and Recurrent Neural Networks?
8. What's the difference between Recurrent Neural Networks and Recursive Neural Networks?
9. When would you use MLP, CNN, and RNN?
10. Why are RNNs (Recurrent Neural Network) better than MLPs at predicting Time Series Data?

- **Mcqs:**

1. RNN Stands for?
a) Recursive Neural Network
b) Recurrent Neural Network
c). Removable Neural Network
d).Recurring Neural Network

2. The main and most important feature of RNN is _____.
a) Visible State
b) Hidden State
c) Present State
d) none of above

3. RNN remembers each and every information through_____
a) Work
b) Time
c) Hours
d) Memory

- **Conclusion :**We have successfully implemented a recurrent neural network to create a classifier.

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from keras.models import Sequential
from keras.layers import Dense,LSTM,Dropout
```

In [2]:

```
data = pd.read_csv('Google_train_data.csv')
data.head()
```

Out[2]:

	Date	Open	High	Low	Close	Volume
0	1/3/2012	325.25	332.83	324.97	663.59	7,380,500
1	1/4/2012	331.27	333.87	329.08	666.45	5,749,400
2	1/5/2012	329.83	330.75	326.89	657.21	6,590,300
3	1/6/2012	328.34	328.77	323.68	648.24	5,405,900
4	1/9/2012	322.04	322.29	309.46	620.76	11,688,800

In [3]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1258 entries, 0 to 1257
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   Date    1258 non-null   object 
 1   Open    1258 non-null   float64
 2   High    1258 non-null   float64
 3   Low     1258 non-null   float64
 4   Close   1258 non-null   object 
 5   Volume  1258 non-null   object 
dtypes: float64(3), object(3)
memory usage: 59.1+ KB
```

In [4]:

```
data["Close"]=pd.to_numeric(data.Close,errors='coerce')
data = data.dropna()
trainData = data.iloc[:,4:5].values
```

In [5]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 1149 entries, 0 to 1257
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   Date    1149 non-null   object 
 1   Open    1149 non-null   float64
 2   High    1149 non-null   float64
 3   Low     1149 non-null   float64
 4   Close   1149 non-null   float64
 5   Volume  1149 non-null   object 
dtypes: float64(4), object(2)
memory usage: 62.8+ KB
```

In [6]:

```
sc = MinMaxScaler(feature_range=(0,1))
trainData = sc.fit_transform(trainData)
trainData.shape
```

Out[6]:

```
(1149, 1)
```

In [7]:

```
X_train = []
y_train = []

for i in range (60,1149): #60 : timestep // 1149 : length of the data
    X_train.append(trainData[i-60:i,0])
    y_train.append(trainData[i,0])

X_train,y_train = np.array(X_train),np.array(y_train)
```

In [8]:

```
X_train = np.reshape(X_train,(X_train.shape[0],X_train.shape[1],1)) #adding the batch_siz
X_train.shape
```

Out[8]:

```
(1089, 60, 1)
```

In [9]:

```
model = Sequential()

model.add(LSTM(units=100, return_sequences = True, input_shape =(X_train.shape[1],1)))
model.add(Dropout(0.2))

model.add(LSTM(units=100, return_sequences = True))
model.add(Dropout(0.2))

model.add(LSTM(units=100, return_sequences = True))
model.add(Dropout(0.2))

model.add(LSTM(units=100, return_sequences = False))
model.add(Dropout(0.2))

model.add(Dense(units =1))
model.compile(optimizer='adam',loss="mean_squared_error")
```

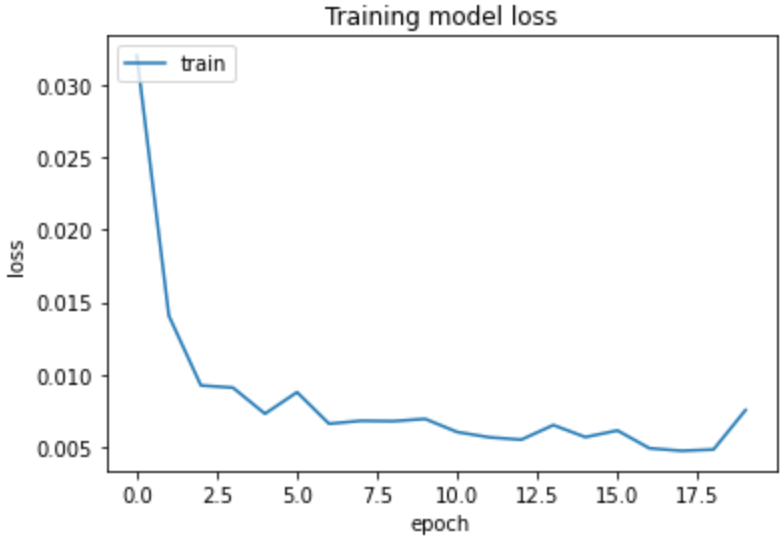
In [10]:

```
hist = model.fit(X_train, y_train, epochs = 20, batch_size = 32, verbose=2)
```

```
Epoch 1/20
35/35 - 16s - loss: 0.0320 - 16s/epoch - 459ms/step
Epoch 2/20
35/35 - 12s - loss: 0.0141 - 12s/epoch - 335ms/step
Epoch 3/20
35/35 - 9s - loss: 0.0093 - 9s/epoch - 244ms/step
Epoch 4/20
35/35 - 7s - loss: 0.0091 - 7s/epoch - 214ms/step
Epoch 5/20
35/35 - 10s - loss: 0.0073 - 10s/epoch - 285ms/step
Epoch 6/20
35/35 - 9s - loss: 0.0088 - 9s/epoch - 250ms/step
Epoch 7/20
35/35 - 7s - loss: 0.0066 - 7s/epoch - 212ms/step
Epoch 8/20
35/35 - 9s - loss: 0.0068 - 9s/epoch - 243ms/step
Epoch 9/20
35/35 - 9s - loss: 0.0068 - 9s/epoch - 253ms/step
Epoch 10/20
35/35 - 7s - loss: 0.0069 - 7s/epoch - 213ms/step
Epoch 11/20
35/35 - 9s - loss: 0.0060 - 9s/epoch - 247ms/step
Epoch 12/20
35/35 - 10s - loss: 0.0057 - 10s/epoch - 284ms/step
Epoch 13/20
35/35 - 7s - loss: 0.0055 - 7s/epoch - 214ms/step
Epoch 14/20
35/35 - 9s - loss: 0.0065 - 9s/epoch - 252ms/step
Epoch 15/20
35/35 - 8s - loss: 0.0057 - 8s/epoch - 242ms/step
Epoch 16/20
35/35 - 7s - loss: 0.0061 - 7s/epoch - 210ms/step
Epoch 17/20
35/35 - 9s - loss: 0.0049 - 9s/epoch - 243ms/step
Epoch 18/20
35/35 - 10s - loss: 0.0047 - 10s/epoch - 282ms/step
Epoch 19/20
35/35 - 7s - loss: 0.0048 - 7s/epoch - 211ms/step
Epoch 20/20
35/35 - 8s - loss: 0.0076 - 8s/epoch - 243ms/step
```

In [11]:

```
plt.plot(hist.history['loss'])
plt.title('Training model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train'], loc='upper left')
plt.show()
```



In [12]:

```
testData = pd.read_csv('Google_test_data.csv')
testData["Close"]=pd.to_numeric(testData.Close,errors='coerce')
testData = testData.dropna()
testData = testData.iloc[:,4:5]
y_test = testData.iloc[60:,0:].values
#input array for the model
inputClosing = testData.iloc[:,0:].values
inputClosing_scaled = sc.transform(inputClosing)
inputClosing_scaled.shape
X_test = []
length = len(testData)
timestep = 60
for i in range(timestep,length):
    X_test.append(inputClosing_scaled[i-timestep:i,0])
X_test = np.array(X_test)
X_test = np.reshape(X_test,(X_test.shape[0],X_test.shape[1],1))
X_test.shape
```

Out[12]:

```
(192, 60, 1)
```

In [13]:

```
y_pred = model.predict(X_test)
y_pred
```

```
6/6 [==============================] - 2s 72ms/step
```

Out[13]:

```
array([[1.16179  ],
       [1.1631088],
       [1.1724331],
       [1.1868168],
       [1.1983689],
       [1.1982452],
       [1.1870549],
       [1.1716752],
       [1.1618347],
       [1.1591649],
       [1.1531832],
       [1.1432477],
       [1.1342392],
       [1.1257831],
       [1.1235965],
       [1.127025 ],
```

In [14]:

```
predicted_price = sc.inverse_transform(y_pred)
```

In [15]:

```
plt.plot(y_test, color = 'red', label = 'Actual Stock Price')
plt.plot(predicted_price, color = 'green', label = 'Predicted Stock Price')
plt.title('Google stock price prediction')
plt.xlabel('Time')
plt.ylabel('Stock Price')
plt.legend()
plt.show()
```